

TaskflowLite

User Guide

Graph • Async • Runtime • Scheduling • Lifetime

任务图 • 异步执行 • 动态运行时 • 调度模型 • 生命周期

VERSION	3.2.0	BASELINE	main / 58eedba09f4e7a9cec5304008007777947efbb59
DATE	2026-09-29	STANDARD	C++20 • Header-only • MIT

31 practical scenarios • 39 repository examples • 50 vector diagrams

CONTENTS

目录

阅读指南	6	9. 任务类型与 callable 协议	19
1. TaskflowLite 是什么	7	10. Branch 与 MultiBranch	20
1.1 当前公开能力	8	10.1 Branch	20
1.2 当前没有作为正式公共 API 提供的能力	10	10.2 MultiBranch	20
2. 三层心智模型	10	10.3 Branch 与汇聚	21
3. 头文件、版本与环境要求	11	11. Jump 与 MultiJump: 强制激活	21
3.1 主入口	11	11.1 单目标 Jump	22
3.2 版本	11	11.2 MultiJump	22
3.3 编译要求	11	11.3 什么时候用 Jump	22
4. CMake 集成	12	11.4 Branch 与 Jump 的选择规则放在一起看	23
4.1 作为子目录	12	12. Module: 嵌套 Flow 与重复执行	23
4.2 安装后使用	12	12.1 定次重复	23
4.3 主要构建选项	12	12.2 谓词控制	24
5. 第一个完整程序: 同步执行 DAG	13	12.3 模块与外层图重复执行	24
6. Flow 与 FlowBuilder	14	12.4 Module 的三个实际用法	24
6.1 所有权	14	13. TaskObject: 把业务对象原地放进 Work	25
6.2 基本构图	15	13.1 原地替换	26
6.3 Placeholder	15	14. Executor: 线程池和提交入口	26
6.4 批量 emplace	15	14.1 三类提交方式	26
6.5 删除与清理	15	14.2 Executor 提交接口怎么选	27
6.6 FlowBuilder 常用接口族	16	15. AsyncFuture: 共享完成状态与结果	28
7. Task 与 TaskView	17	15.1 常用状态	29
7.1 Task 是可空非拥有句柄	17	15.2 wait 与 get 的区别	29
7.2 TaskView 是只读视图	17	15.3 停止	29
7.3 替换 callable, 但保留节点身份	17	15.4 get() 对不同结果类型的返回	30
7.4 Task 的四类操作	18	16. AsyncTask: 先配置, 再启动一次	30
8. 建边: checked 与 unchecked	18	16.1 状态规则	30
8.1 默认 checked	18	16.2 start 的依赖规则	31
8.2 显式跳过检查	19	16.3 AsyncTask 生命周期	31

17. 动态异步依赖到底保证什么	32	27.1 普通任务异常	44
17.1 为什么依赖会保留前驱生命周期	33	27.2 Observer 异常	45
17.2 跨 Executor	33	27.3 TaskGroup	45
17.3 多前驱依赖的运行顺序	33	27.4 异常从哪里观察	45
18. AsyncTaskObject：延迟异步任务 + 原地对象	34	28. ScopedExceptionAnchor：显式局部异常边界	46
19. Executor 的 Graph 提交重载	34	29. Semaphore：任务级资源配额	47
19.1 async	34	29.1 多 Semaphore 是 all-or-nothing	47
19.2 silent_async	35	29.2 reset	48
19.3 defer_async	35	29.3 生命周期	48
20. corun 与 wait_for_all	35	29.4 多资源获取示例	48
20.1 Executor::corun	35	30. TaskObserver：任务执行观察	49
20.2 wait_for_all	36	31. WorkerHandler：线程生命周期钩子	50
20.3 选择	36	30.3 一个可计时的 TaskObserver	51
21. Context：执行期统一上下文	36	31.3 WorkerHandler 的典型用途	51
22. Runtime：任务运行时动态派生	37	32. D2 图导出与命名	52
22.1 Runtime 的子任务归属	37	32.3 导出到文件	52
22.2 run(graph)	37	33. 并行模式一：fan-out / fan-in	53
22.3 corun(graph)	38	34. 并行模式二：索引分块	54
22.4 wait_until	38	35. 并行模式三：Pipeline、生产者/消费者和状态机	55
22.5 Runtime 常用接口表	38	35.1 Pipeline	55
23. SubFlow：执行期间动态构图	39	35.2 Producer / Consumer	55
23.1 run 与 wait	39	35.3 状态机 / Retry	55
23.3 动态数量节点示例	40	35.3 一个完整的三阶段 Pipeline	56
24. TaskGroup：局部分组、停止与异常边界	40	35.4 Map-Reduce 模式	57
24.1 TaskGroup 当前语义	40	36. Executor 视角：一次图执行发生什么	57
24.2 run 与 corun	41	36.1 _set_up_graph	58
24.3 独立于父停止域的任务	41	36.2 cache 接力	58
24.4 TaskGroup 的局部批处理	41	36.4 corun 在外部线程和 Worker 内部的差异	58
25. 为什么 Worker 内要协作等待	42	37. 本地队列与 SharedWorkStack	58
26. 协作式停止	43	37.1 Worker 本地 BoundedQueue	58
26.1 停止沿父 Topology 链继承	44	37.2 队列满时	59
26.2 停止不会自动做的事	44	37.3 SharedWorkStack	59
27. 异常传播模型	44	38. Work stealing 与 Notifier	59

38.1 为什么必须两阶段	60	47.2 “单线程卡住”	68
38.2 当前 Notifier 内部	60	47.3 “依赖前驱抛异常了，后继为什么还可能被调度”	69
39. Work、Graph 与边表布局	60	47.4 “TaskGroup 里抛异常，离开作用域没有 catch 到”	69
39.1 两种 parent 不要混淆	61	47.5 “Semaphore reset 失败”	69
40. join_counter：依赖到达与 parent slot 守恒	61	47.6 “图第二次运行结果不对”	69
40.1 静态 strong dependency	61	48. 内存分配失败与强保证边界	69
40.2 执行完成后恢复静态 weight	62	49. 测试、Sanitizer 与 CI	70
40.3 parent slot	62	49.1 ASan	70
40.4 两前驱汇聚的计数时间线	62	49.2 TSan	70
41. Branch、Jump 在 tear-down 阶段的区别	63	49.3 Header 独立编译	70
42. 异步动态依赖的内部协议	64	49.4 no-exception 测试	71
43. 内存、对象和结果存储	64	50. Benchmark 方法	71
43.1 Work 对象池	64	51. 完整实战示例集	72
43.2 SmallVector	65	51.1 示例 1：最小但完整的 fan-out / fan-in DAG	72
43.3 ResultSlot	65	51.2 示例 2：批量 emplace + linearize	73
43.4 callable / object 原地存储	65	51.3 示例 3：Placeholder 先占拓扑，稍后绑定执行体	73
43.5 Async Work 生命周期图	65	51.4 示例 4：Branch 实现真正的 if/else 路由	74
44. 禁用异常	66	51.5 示例 5：MultiBranch 同时广播到多个后继	75
44.1 关闭后的语义	66	51.6 示例 6：Jump 构造受控 retry 循环	75
45. Debug 检查和关键宏	66	51.7 示例 7：Module 把成熟子流程复用 3 次	76
45.1 TFL_ENABLE_ASSERT	66	51.8 示例 8：TaskObject 保存不可移动业务状态	77
45.2 TFL_ENABLE_WORK_EXECUTION_CHECK	67	51.9 示例 9：保留拓扑，只替换对象实现	77
45.3 TFL_CACHE_LINE_SIZE	67	51.10 示例 10：async 返回值与重复 get()	78
45.4 TFL_ENABLE_TASK_POOL	67	51.11 示例 11：为什么 wait() 后仍然要 get()	78
46. 线程安全与生命周期清单	67	51.12 示例 12：defer_async 先创建、配置、再启动	78
46.1 Flow	67	51.13 示例 13：动态依赖链 Extract -> Transform -> Load	79
46.2 Task / TaskObject	67	51.14 示例 14：AsyncFuture 钻石依赖	79
46.3 AsyncFuture / AsyncTask	67	51.15 示例 15：跨 Executor 的完成依赖	80
46.4 Runtime / SubFlow / TaskGroup / Branch / Jump	68	51.16 示例 16：AsyncTaskObject 同时保存任务状态和结果	80
46.5 Semaphore	68	51.17 示例 17：Runtime 动态 fan-out，然后协作等待	81
46.6 Observer / Handler	68	51.18 示例 18：Runtime 中同步执行一个已有子图	82
47. 常见错误与排查	68	51.19 示例 19：wait_until 等动态 Future 集合	82
47.1 “任务没有执行”	68	51.20 示例 20：SubFlow 根据输入动态构图	83

51.21 示例 21: TaskGroup 管理局部批次	84
51.22 示例 22: 协作式停止	85
51.23 示例 23: Semaphore 把并行任务限制为固定资源数	86
51.24 示例 24: 一个任务同时需要 GPU + 内存池	86
51.25 示例 25: Observer 做轻量任务计数	87
51.26 示例 26: WorkerHandler 初始化每个 Worker 线程	87
51.27 示例 27: 手工 parallel-for 分块	88
51.28 示例 28: 无锁共享写的 Map-Reduce	88
51.29 示例 29: Retry + backoff 状态机	89
51.30 示例 30: 运行之间替换算法, 但不重新构图	90
51.31 示例 31: 完整视觉处理 Pipeline	90
52. 39 个示例阅读路线	92
53. 公共 API 速查	94
53.1 Flow / FlowBuilder	94
53.2 Task / TaskView	95
53.3 Branch / MultiBranch / Jump / MultiJump	95
53.4 Executor	96
53.5 AsyncFuture	96
53.6 AsyncTask	96
53.7 TaskObject / AsyncTaskObject<R,T>	97
53.8 Context / Runtime	97
53.9 SubFlow	98
53.10 TaskGroup	98
53.11 Semaphore	98
53.12 TaskObserver / WorkerHandler	98
53A. 常用场景配方	99
配方 1: 两个前处理并行, 一个后处理汇聚	99
配方 2: 异步任务链	99
配方 3: 单 Worker 内安全等待子任务	99
配方 4: 资源受限任务	99
配方 5: 局部任务组	100
配方 6: 动态数量 SubFlow	100
配方 7: 可复用对象节点	100

配方 8: 同步嵌套子图	100
54. 当前 main 相对旧 PDF 必须更新的重点	100
55. 设计和使用原则总结	101
56. 源码索引	101
57. 文档维护规则	102
附录 A / 一个较完整的组合示例	102
附录 B / 选择接口的决策表	104

Public API • Graph • Async • Runtime • Scheduling • Lifetime

公开接口 • 任务图 • 异步执行 • 动态运行时 • 调度模型 • 生命周期

- 适用版本: **TaskflowLite 3.2.0**
- 源码基线: [wicyn/taskflowlite main](#)
- 核对提交: [58eedba09f4e7a9cec5304008007777947efbb59](#)
- 核对日期: **2026-09-29**
- 语言标准: **C++20**
- 许可证: **MIT**

本手册以当前 `main` 分支的 `taskflowlite/core`、入口头文件、CMake、测试与 01-39 示例为准，并额外提供 31 组按当前公开 API 编写的实战场景。当历史文档、旧版 PDF 或旧示例与当前代码不一致时，以当前源码实现和测试契约为准。

阅读指南

建议先掌握 **Flow/Task → Executor → Async/Runtime → 控制与生命周期**，再阅读调度器内部章节。

TaskflowLite 的公共接口不多，但它同时覆盖静态任务图、动态任务、异步结果、条件控制流、资源限流、停止、异常与工作窃取。最容易出错的地方不是 API 记忆，而是**生命周期、依赖语义、协作等待和控制流语义**。

推荐阅读路线：

1. 新用户：01 → 04 → 05 → 08 → 14 → 20。
2. 异步任务：14 → 15 → 16 → 17 → 18。
3. 动态任务：21 → 22 → 23 → 24 → 25。
4. 条件与循环：10 → 11 → 12。
5. 维护调度器：36 → 37 → 38 → 39 → 40 → 41。
6. 排查异常/卡住：26 → 27 → 28 → 42。
7. 直接看代码：51（31 组实战）→ 52（仓库 01-39 示例路线）。

图示约定

本文所有结构图使用同一套技术手册视觉语言：**蓝色**表示执行/调度入口，**青色**表示 Graph/Work 等核心对象，**紫色**表示 Topology/异步状态，**绿色**表示 ready/完成，**橙色**只用于条件判断、资源配额等控制语义。节点统一使用圆角矩形；主关系使用细实线箭头；继承、回流或辅助关系使用虚线；复杂回路才使用弧线，避免无意义的折线和交叉。

文档中的术语

- **静态图**：由 `Flow` 拥有的 `Graph + Work` 集合。

- **动态任务**：运行期间由 `Runtime`、`TaskGroup` 等创建的异步 Work。
- **强依赖**：普通任务、Branch 等按照 `join_counter` 到达协议参与的依赖。
- **Jump 激活**：通过 Jump/MultiJump 强制激活目标，不等价于普通强依赖到达。
- **协作等待**：Worker 等待时继续执行其他 ready Work，而不是把工作线程阻塞在条件变量或 Future 上。
- **Topology**：一次异步执行的完成、停止、状态和引用生命周期控制域。
- **parent slot**：父 Work 的未完成计数槽，用于保证子工作完成前父工作不能结束。

1. TaskflowLite 是什么

TaskflowLite（命名空间 `tfl`）是一个仅头文件、C++20、面向任务图的并行调度库。应用程序描述：

- 有哪些工作；
- 工作之间有什么依赖；
- 哪些路径由运行时条件选择；
- 哪些工作可以动态生成；
- 哪些任务必须受共享资源容量限制。

Executor 负责：

- 创建固定数量 Worker；
- 发布 ready Work；
- Worker 本地 LIFO 执行；
- 从其他 Worker 或共享分片窃取工作；
- 没工作时退避并进入 Notifier 两阶段休眠；
- 完成时传播依赖、异常、停止与父子完成计数。

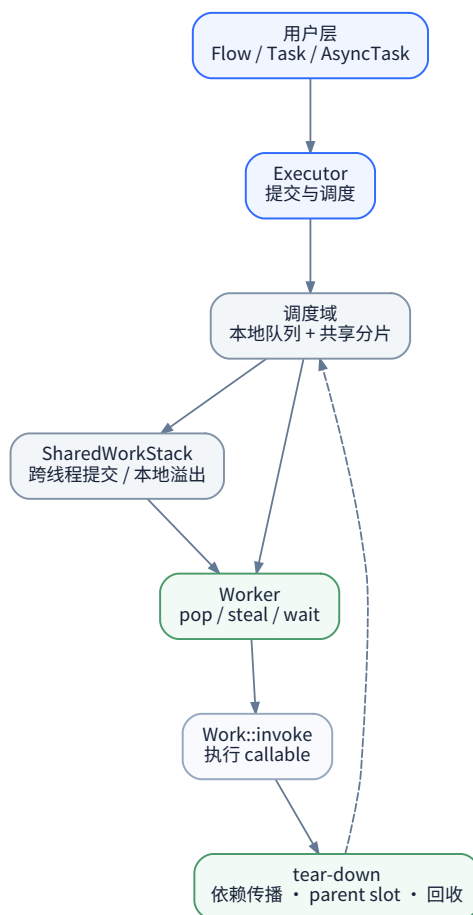


图 1 • TaskflowLite 从用户任务图到 Executor 调度与 Work 收尾的主路径

1.1 当前公开能力

类别	当前能力
静态任务图	Flow、Task、FlowBuilder、模块子图
条件控制	Branch、MultiBranch
强制跳转/循环	Jump、MultiJump
即时异步	Executor::async、silent_async
延迟启动	Executor::defer_async + AsyncTask::start
异步依赖	AsyncFuture / AsyncTask 作为动态前置依赖
动态执行	Runtime
动态子图	SubFlow
局部分组	TaskGroup
同步图执行	Executor::corun

类别	当前能力
协作停止	<code>request_stop</code> / <code>stop_requested</code>
资源限流	<code>Semaphore</code>
原地业务对象	<code>TaskObject</code> / <code>AsyncTaskObject</code>
观察与线程生命周期	<code>TaskObserver</code> / <code>WorkerHandler</code>
可视化	D2 文本导出
异常模式	异常传播或 <code>TFL_ENABLE_EXCEPTIONS=0</code>
内部调度	本地 <code>BoundedQueue</code> + 分片 <code>SharedWorkStack</code> + work stealing + <code>Notifier</code>

1.1.1 按需求选择入口

你要做什么	首选入口	返回/等待方式	关键约束
构建固定 DAG	<code>Flow + Task</code>	<code>executor.corun(flow)</code>	执行期间不修改图
提交一次有结果任务	<code>Executor::async</code>	<code>AsyncFuture::get()</code>	<code>get()</code> 才传播任务异常
先配置再启动	<code>Executor::defer_async</code>	<code>AsyncTask::start()</code>	同一底层任务只能成功启动一次
运行中派生子任务	<code>Runtime::async</code>	<code>Runtime::wait/wait_until</code>	使用协作等待
运行中动态构图	<code>SubFlow</code>	<code>run() + wait()</code>	只构图不会自动执行
在一个作用域管理一组任务	<code>TaskGroup</code>	<code>group.wait()</code>	析构只等待，不传播子任务异常
同步执行一个图	<code>corun</code>	调用返回即完成	Worker 内自动协作执行
控制资源并发	<code>Task::acquire/release</code> + <code>Semaphore</code>	框架挂起/重新发布 Work	不阻塞 Worker
保留业务对象状态	<code>emplace_object/defer_async_object</code>	<code>object()</code>	<code>object()</code> 本身不做同步



图 2 · 按使用需求选择 TaskflowLite 入口

1.2 当前没有作为正式公共 API 提供的能力

当前 `core` 没有独立的 STL 风格并行算法层，例如不存在正式发布的：

```
parallel_for
for_each_index
reduce
transform_reduce
scan
sort
merge
```

仓库示例会用任务图、Runtime、TaskGroup 展示并行循环、归约、Pipeline 等模式。手册后文会介绍这些**现有 API 能实现的模式**，不会把未来规划写成已经发布的接口。

2. 三层心智模型

理解 TaskflowLite 最重要的是把“图”“执行”和“调度器内部”分开。

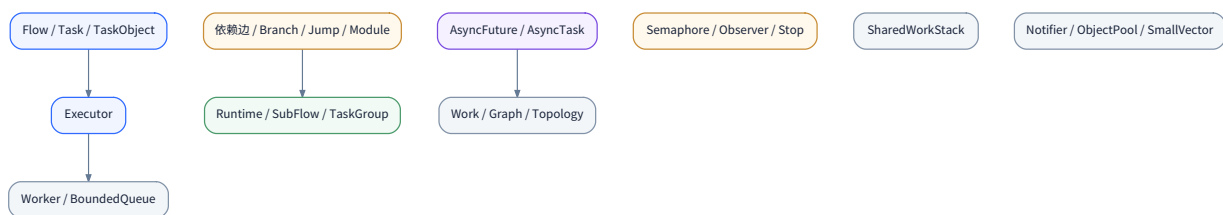


图 3 • TaskflowLite 的三层心智模型

描述层回答“做什么”

`Flow` 拥有静态节点，`Task` 只是借用句柄。依赖边定义业务顺序，不指定具体线程。

执行控制层回答“什么时候执行、怎么等待”

`Executor` 是线程池和调度器；`AsyncFuture` 表示共享完成状态；`AsyncTask` 表示尚可启动一次的延迟任务；`Runtime` 等用于 Worker 内部动态派生工作。

内部调度层回答“ready Work 去哪里”

ready Work 可能：

1. 直接通过 `cache` 在当前 Worker 连续执行；
2. 进入当前 Worker 本地队列；
3. 本地队列满后溢出共享分片；
4. 外部线程直接发布到共享分片；

5. 被其他 Worker 窃取。

3. 头文件、版本与环境要求

3.1 主入口

绝大多数应用只需：

```
#include <taskflowlite/taskflowlite.hpp>
```

当前入口包含 `Task`、`AsyncFuture`、`AsyncTask`、`TaskObject`、`AsyncTaskObject`、`TaskGroup`、`Flow`、`Branch`、`Jump`、`Runtime`、`SubFlow`、`Executor` 和 `Work` 工厂。

显式异常锚点当前不是主入口直接包含项，使用时建议明确包含：

```
#include <taskflowlite/core/scoped_exception_anchor.hpp>
```

3.2 版本

```
#include <taskflowlite/taskflowlite.hpp>
#include <iostream>

int main() {
    std::cout << tfl::version << '\n';    // 3.2.0
    std::cout << tfl::version.to_string(); // "3.2.0"
}
```

当前源码宏：

```
TASKFLOWLITE_VERSION_MAJOR
TASKFLOWLITE_VERSION_MINOR
TASKFLOWLITE_VERSION_PATCH
```

3.3 编译要求

- C++20;
- CMake >= 3.21;
- 标准库需要支持当前代码使用的 `std::format`、原子等待等功能；
- GCC 路线建议使用 GCC 13+ 配套 libstdc++;
- Windows 由 CMake target 自动补 `/utf-8`、`/Zc:__cplusplus`;
- 非 MSVC、非 Apple 平台由导出 target 链接 `atomic`;

- 核心库没有第三方运行时依赖。

4. CMake 集成

4.1 作为子目录

```
cmake_minimum_required(VERSION 3.21)
project(my_app LANGUAGES CXX)

add_subdirectory(third_party/taskflowlite)

add_executable(my_app main.cpp)
target_link_libraries(my_app PRIVATE TaskflowLite::taskflowlite)
```

TaskflowLite 是 `INTERFACE` 头文件库，目标会传递 C++20、线程库、包含目录和平台链接要求。

4.2 安装后使用

安装：

```
cmake -S . -B build/install \
  -DTFL_BUILD_EXAMPLES=OFF \
  -DTFL_BUILD_TESTS=OFF \
  -DCMAKE_INSTALL_PREFIX=install

cmake --install build/install
```

下游：

```
find_package(TaskflowLite CONFIG REQUIRED)

add_executable(app main.cpp)
target_link_libraries(app PRIVATE TaskflowLite::taskflowlite)
```

通过 `CMAKE_PREFIX_PATH` 指向安装前缀。

4.3 主要构建选项

选项	默认	用途
<code>TFL_BUILD_EXAMPLES</code>	顶层项目 ON	构建示例
<code>TFL_BUILD_TESTS</code>	OFF	构建测试；可能获取 Catch2

选项	默认	用途
<code>TFL_BUILD_BENCHMARKS</code>	OFF	构建基准；对比项需要 Taskflow
<code>TFL_BUILD_DOCS</code>	OFF	生成 Doxygen
<code>TFL_NATIVE_ARCH</code>	OFF	Release 内部目标按本机构架优化
<code>TFL_SANITIZER</code>	OFF	<code>OFF</code> / <code>ASAN</code> / <code>TSAN</code>

测试默认关闭，避免普通消费项目 configure 时隐式下载测试依赖。

5. 第一个完整程序：同步执行 DAG

当前最直接的“构图后等它完成”写法是 `Executor::corun`。

```
#include <taskflowlite/taskflowlite.hpp>
#include <iostream>

int main() {
    tfl::Executor executor(4);
    tfl::Flow flow{"basic"};

    int left = 0;
    int right = 0;
    int result = 0;

    auto a = flow.emplace([&] { left = 20; }).name("left");
    auto b = flow.emplace([&] { right = 22; }).name("right");
    auto c = flow.emplace([&] {
        result = left + right;
    }).name("sum");

    a.precede(c);
    b.precede(c);

    executor.corun(flow);

    std::cout << result << '\n';
    return result == 42 ? 0 : 1;
}
```

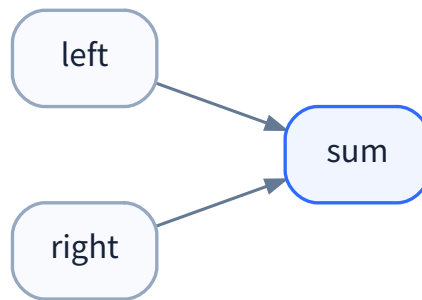


图 4 • 最小 DAG：两个并行前驱汇聚到一个后继

a 和 b 可以并行；c 必须等两个强前驱都到达后才 ready。

corun 的关键语义

- 外部线程调用：提交图并阻塞当前外部线程，直到该图完成；
- Executor 自己的 Worker 调用：采用协作执行，不把 Worker 单纯阻塞；
- 图内异常会在 corun 返回边界重新抛出；
- 本次等待范围只对应这次 corun 建立的完成锚点，不是“整个 Executor 的全局停止世界”。

6. Flow 与 FlowBuilder

Flow 是静态图所有者，同时继承 FlowBuilder 的构图接口。

6.1 所有权

```

Flow
├── owns Graph
│   └── owns Work nodes
│       ↑
│       Task / TaskObject 仅借用
  
```

因此：

- Flow 不可复制；
- Flow 可移动，但移动时必须没有执行中的引用；
- Task 复制只是复制节点指针；
- Flow::erase / clear 后，对应 Task、TaskObject::object() 引用都会悬空；
- 图执行期间不得修改、移动、清空或销毁。

6.2 基本构图

```
tfl::Flow flow;

auto a = flow.emplace([] {});
auto b = flow.emplace([] {});
auto c = flow.emplace([] {});

flow.linearize(a, b, c);
```

等价于：

```
a.precede(b);
b.precede(c);
```

也支持范围：

```
std::array<tfl::Task, 3> tasks{a, b, c};
flow.linearize(tasks);
```

6.3 Placeholder

```
auto node = flow.placeholder();
node.name("configured later");

node.work([] {
    // 真正执行体
});
```

Placeholder 适合先构拓扑、后绑定 callable。

6.4 批量 emplace

当前 `FlowBuilder::emplace(Ts...)` 可以按参数顺序混合插入 callable、graph holder 和 `tfl::pack`。

需要注意：**批量插入不是事务**。后面的节点构造失败时，前面已经插入的节点不会自动回滚。

6.5 删除与清理

```
flow.erase(task);
flow.erase(a, b, c);
flow.clear();
```

这些操作只能在图静止时进行。

6.6 FlowBuilder 常用接口族

`Flow` 和 `SubFlow` 都复用 `FlowBuilder`，因此下面这些构图操作在静态图和动态子图中保持一致。

接口	典型用途	返回值
<code>placeholder()</code>	先建结构，后绑定执行体	<code>Task</code>
<code>emplace(callable)</code>	Basic / Branch / Jump / Runtime / SubFlow 节点	<code>Task</code>
<code>emplace(graph, ...)</code>	Module 节点	<code>Task</code>
<code>emplace_object<T>(args...)</code>	在 Work 内原地构造业务对象	<code>TaskObject<T></code>
<code>emplace(a, b, ...)</code>	批量创建多个节点	<code>std::tuple<...></code>
<code>linearize(...)</code>	把一组任务串成线性依赖	<code>void</code>
<code>erase(...)</code> / <code>clear()</code>	静止状态下删除节点/清空图	<code>void</code>
<code>for_each(visitor)</code>	遍历当前图中的任务句柄	<code>void</code>
<code>graph()</code>	获取底层 Graph 引用	<code>Graph&</code>

一次创建多个任务

```
auto [load, decode, infer, save] = flow.emplace(
    [&] { load_frame(); },
    [&] { decode_frame(); },
    [&] { run_inference(); },
    [&] { save_result(); }
);

flow.linearize(load, decode, infer, save);
```

同一 Flow 完成后复用

```
tfl::Flow flow;
int calls = 0;
auto task = flow.emplace([&] { ++calls; });

executor.corun(flow);
executor.corun(flow);

assert(calls == 2);
```

可复用的是**图结构**；callable 捕获的业务状态不会自动重置。

7. Task 与 TaskView

7.1 Task 是可空非拥有句柄

常用查询：

```
task.valid();
task.name();
task.type();
task.num_predecessors();
task.num_successors();
task.num_acquires();
task.num_releases();
task.num_observers();
task.hash_value();
task.has_exception_ptr();
task.exception_ptr();
```

`valid()` 只说明内部 `Work* != nullptr`，不能检测悬空指针。

7.2 TaskView 是只读视图

`TaskView` 用于 Branch 谓词、只读遍历等场景，可以查询节点但不能修改图。

```
const tfl::Task task = ...;

task.for_each_successor([](tfl::TaskView view) {
    std::cout << view.name() << '\n';
});
```

7.3 替换 callable，但保留节点身份

```
tfl::Flow flow;
int value = 0;

auto task = flow.placeholder();
task.name("editable");

task.work([&] { value = 10; });
executor.corun(flow);

task.work([&] { value += 32; });
executor.corun(flow);
```

`work(...)` 只替换 Payload；节点身份、名称、边关系、Semaphore 配置和 Observer 仍属于同一个 Work。

当前回归测试还覆盖了：如果替换对象构造本身抛异常，原 callable 与图结构应保持可用。

7.4 Task 的四类操作

类别	代表接口	是否修改节点
查询	<code>valid/name/type/num_* / exception_ptr</code>	否
拓扑	<code>precede/succeed/remove_*/clear_*</code>	是
执行体	<code>work/work_object</code>	是
资源/诊断	<code>acquire/release/register_observer/dump</code>	是

遍历邻接任务

```
task.for_each_successor([](tfl::Task next) {
    std::cout << next.name() << '\n';
});

const tfl::Task snapshot = task;
snapshot.for_each_predecessor([](tfl::TaskView prev) {
    std::cout << prev.name() << '\n';
});
```

保留拓扑，只替换执行对象

```
struct Counter {
    int value{};
    void operator()() { ++value; }
};

auto node = flow.placeholder();
auto sink = flow.emplace([] {});
node.precede(sink);

Counter& counter = node.work_object<Counter>();
executor.corun(flow);
assert(counter.value == 1);
```

`work()` / `work_object()` 不改变 Work 身份、名称、已有边、Semaphore 配置和 Observer；替换失败时当前实现保留旧 payload，图仍可继续使用。

8. 建边：checked 与 unchecked

8.1 默认 checked

```
a.precede(b);
b.succeed(a);
```

默认检查空句柄、跨图、重复边以及非法闭环等构图错误。

8.2 显式跳过检查

```
a.precede<false>(b);
b.succeed<false>(a);
flow.linearize<false>(a, b, c);
```

这适合：

- 图生成器已经在上层验证过拓扑；
- 构图性能确实重要；
- 调用者能保证节点有效、同图、无重复和非法闭环。

`<false>` 不表示无分配，边表扩容仍可能失败；参数包逐边插入时也不提供整个操作的事务回滚。

9. 任务类型与 callable 协议

当前 `TaskType`：

TaskType	用户 callable 形式	调度语义
Placeholder	无	只传播依赖
Basic	<code>f()</code>	普通任务
Branch	<code>f(Branch&)</code>	选择一个后继
MultiBranch	<code>f(MultiBranch&)</code>	选择多个后继
Jump	<code>f(Jump&)</code>	强制激活一个目标
MultiJump	<code>f(MultiJump&)</code>	强制激活多个目标
Runtime	<code>f(Runtime&)</code>	执行期间派生工作
Graph	graph holder	执行嵌套子图

`SubFlow` callable 也是公共构图方式：

```
flow.emplace([](tfl::SubFlow& sf) {
    ...
});
```

`SubFlow` 最终对应动态 Graph 语义。

捕获规则

框架保存 callable 的衰减类型：

- 左值通常复制；
- 右值移动/转发；
- 需要显式借用外部对象时可使用 `std::ref`；
- 所有借用对象必须活到任务最后一次访问完成。

10. Branch 与 MultiBranch

Branch 是**条件路由**，不是“忽略其他前驱的强制跳转”。

10.1 Branch

```
auto route = flow.emplace([&](tfl::Branch& br) {
    br.select(use_fast ? 0 : 1);
});

auto fast = flow.emplace([] {});
auto safe = flow.emplace([] {});

route.precede(fast, safe);
```

也可以：

```
br(0);
br[1] = true;
br.reset();
br.select_if([](tfl::TaskView view) {
    return view.name() == "fast";
});
```

无有效选择时，本次 Branch 不传播任何后继。

10.2 MultiBranch

```
auto route = flow.emplace([](tfl::MultiBranch& br) {
    br.select(0, 2);
});

route.precede(a, b, c);
```

常用：

```
br.select(0, 1);  
br.unselect(1);  
br.select_all();  
br.unselect_all();  
br.select_if(...);  
br[2] = true;
```

选择集合自动去重。

10.3 Branch 与汇聚

如果目标还有其他强前驱，Branch 只是为被选目标提供一次**正常 strong arrival**；目标仍需满足自己的全部 join 条件。

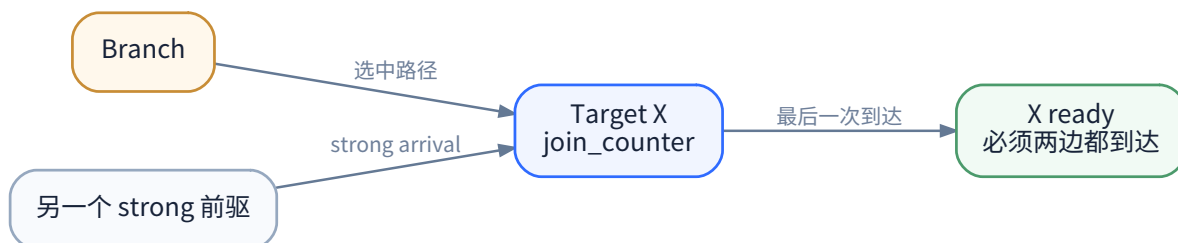


图 5 • Branch 仍遵守目标节点的 strong dependency join

11. Jump 与 MultiJump: 强制激活

Jump 的关键不是“选路径”三个字，而是：它会绕过目标普通 strong join 屏障，强制把目标带入可执行状态。



图 6 • Branch 与 Jump 对目标节点就绪语义的差异

11.1 单目标 Jump

```
auto jump = flow.emplace([&](tfl::Jump& j) {  
    if (retry) {  
        j.select(0);  
    } else {  
        j.select(1);  
    }  
});
```

接口与 Branch 类似：

```
j.select(index);  
j(index);  
j[index] = true;  
j.unselect(index);  
j.reset();  
j.select_if(...);
```

11.2 MultiJump

```
mj.select(0, 2, 3);  
mj.unselect(2);  
mj.select_all();  
mj.reset();
```

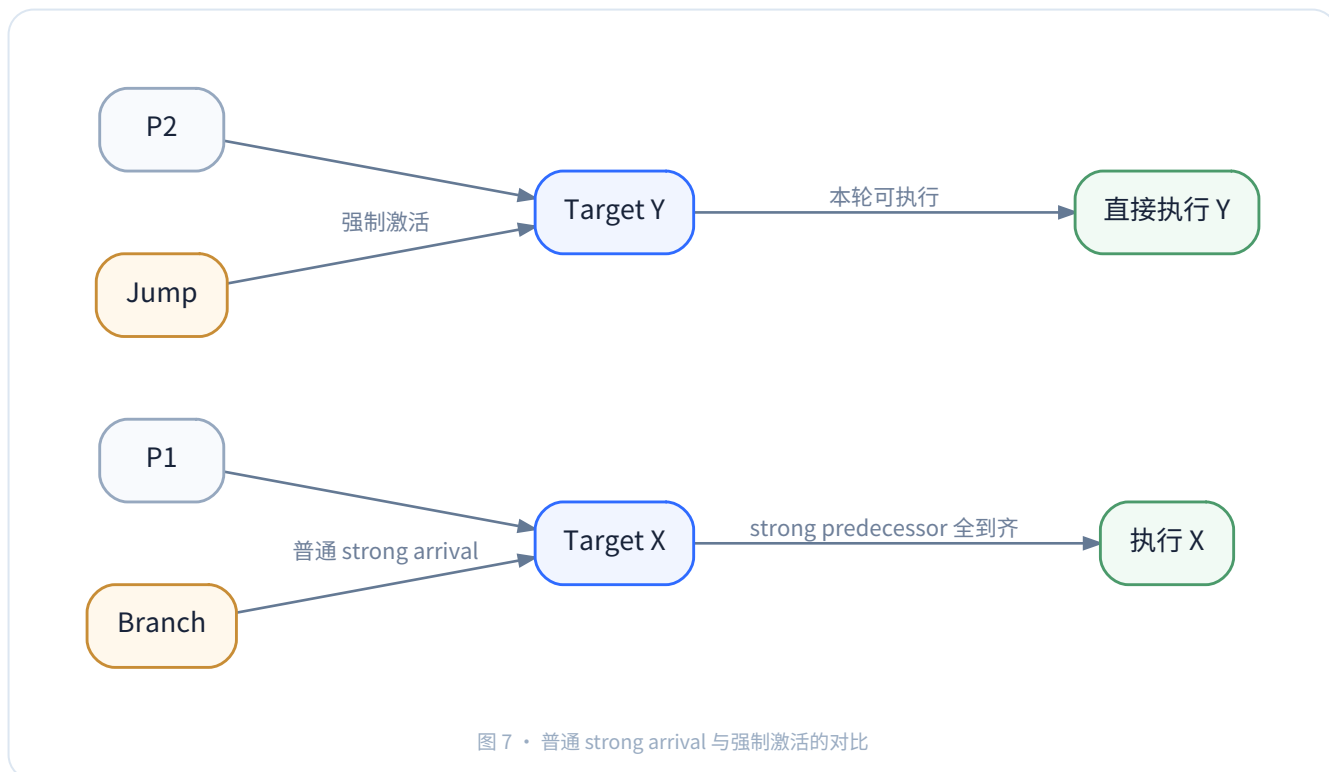
11.3 什么时候用 Jump

适合：

- 状态机；
- retry/backoff；
- 受控循环；
- 明确知道“此路径本轮必须重新激活”的控制流。

不要把普通 DAG 中的条件分支全部替换成 Jump。Jump 改变了依赖屏障语义，应当作为显式控制流原语使用。

11.4 Branch 与 Jump 的选择规则放在一起看



因此：

- **Branch** 适合 `if/else`、路由、策略选择；
- **Jump** 适合重试、状态机回跳、受控循环；
- 若目标还需要其他正常前驱全部完成，不应使用 `Jump` 代替 `Branch`。

12. Module：嵌套 Flow 与重复执行

一个 `Flow` 可以作为另一个图中的 Graph 节点。

```
tfl::Flow child;
child.emplace([] { /* child work */ });

tfl::Flow outer;
auto module = outer.emplace(child);
```

左值子图按借用方式保存，因此必须存活到模块最后一次执行完成。

12.1 定次重复

```
auto module = outer.emplace(child, 3);
```

表示最多执行子图 3 次。空子图、停止或异常都可能让实际次数提前结束。

12.2 谓词控制

```
int rounds = 0;

auto module = outer.emplace(child, [&] {
    return rounds++ >= 5;    // true 表示停止
});
```

谓词语义统一为：**true 停止，false 继续下一轮。**

12.3 模块与外层图重复执行

模块内部计数状态会按其实现约定在正常完成后恢复，使外层 Flow 下一次执行时可以再次运行预期轮数。业务对象本身的成员状态不会自动清零，是否重置由应用定义。

12.4 Module 的三个实际用法

把成熟子流程当作一个节点

```
tfl::Flow decode("decode");
auto d0 = decode.emplace([] { read_header(); });
auto d1 = decode.emplace([] { decode_body(); });
d0.precede(d1);

tfl::Flow main;
auto input = main.emplace([] { open_file(); });
auto module = main.emplace(decode);
auto output = main.emplace([] { publish(); });
main.linearize(input, module, output);
```

固定重复 N 次

```
auto warmup = main.emplace(decode, 5);
```

由谓词决定是否继续

```
std::atomic<bool> done{false};
auto loop = main.emplace(decode, [&] {
    return done.load(std::memory_order_acquire);
});
```

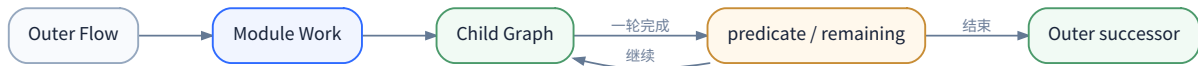


图 8 • Module 节点驱动物子图重复执行

Module 负责复用子图结构；如果需要在运行期间按输入动态创建不同节点，应使用 `SubFlow`。

13. TaskObject: 把业务对象原地放进 Work

如果 callable 是一个有状态对象，并且希望：

- 避免临时对象；
- 不要求可复制/可移动；
- 在图外访问内部状态；

使用 `emplace_object<T>`。

```

struct Counter {
    int calls{0};

    void operator()() {
        ++calls;
    }
};

tfl::Flow flow;
auto task = flow.emplace_object<Counter>();

executor.corun(flow);

std::cout << task.object().calls << '\n';
  
```

`TaskObject<T>` 继承自 `Task`，因此仍可：

```

task.name("counter");
task.precede(next);
task.acquire(semaphore);
  
```

生命周期

`object()` 返回对 Work 内部对象的直接引用：

- 不复制；
- 不增加生命周期；
- 节点销毁、被 `erase/clear` 或 callable 被替换后引用失效；
- 与任务并发访问时由应用负责同步。

13.1 原地替换

普通 `Task` 也支持：

```
auto& object = task.work_object<Counter>();
```

同样保留节点身份和拓扑，只替换执行对象。

14. Executor：线程池和提交入口

```
tfl::Executor executor;  
tfl::Executor executor4(4);
```

也可以绑定 `WorkerHandler`：

```
MyHandler handler;  
tfl::Executor executor(handler, 4);
```

Executor 不拥有 handler，handler 必须活过整个 Executor。

14.1 三类提交方式

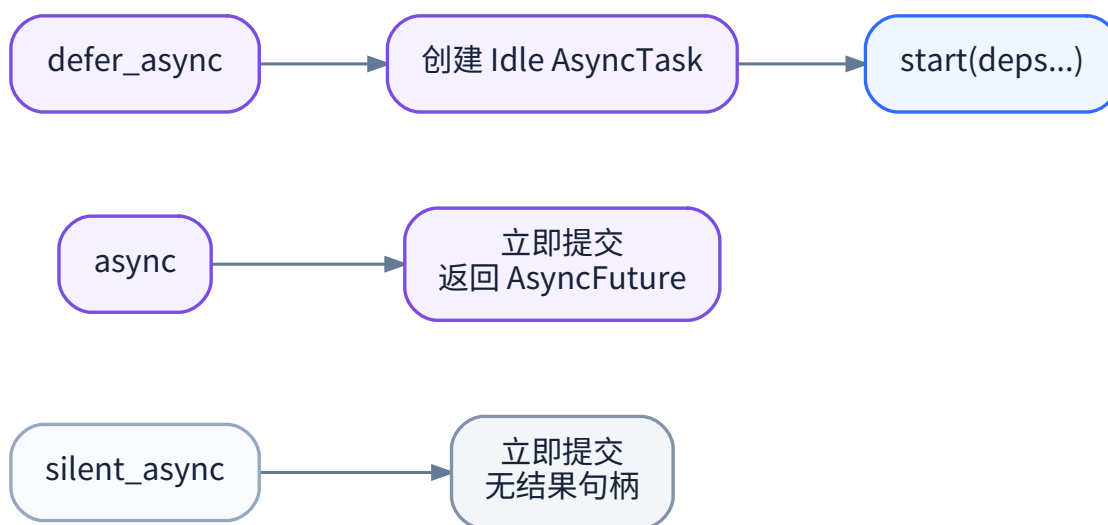


图 9 • Executor 三种提交模式及返回形式

Fire-and-forget

```
executor.silent_async([] {
    // 无结果句柄
});
```

立即异步

```
auto future = executor.async([] {
    return 42;
});

std::cout << future.get();
```

延迟任务

```
auto task = executor.defer_async([] {
    return 42;
});

task.name("deferred");
task.start();

std::cout << task.get();
```

14.2 Executor 提交接口怎么选

接口	是否立即提交	是否返回结果句柄	典型用途
<code>silent_async</code>	是	否	fire-and-forget
<code>async</code>	是	<code>AsyncFuture<R></code>	普通异步计算/依赖
<code>defer_async</code>	否	<code>AsyncTask<R></code>	先配置 name/ Semaphore/Observer/ 依赖
<code>defer_async_object<T></code>	否	<code>AsyncTaskObject<R, T></code>	原地业务对象 + 延迟启动
<code>corun(graph)</code>	是	同步返回	同步执行一张图
<code>wait_for_all()</code>	-	-	等当前观察到的顶层 topology 归零

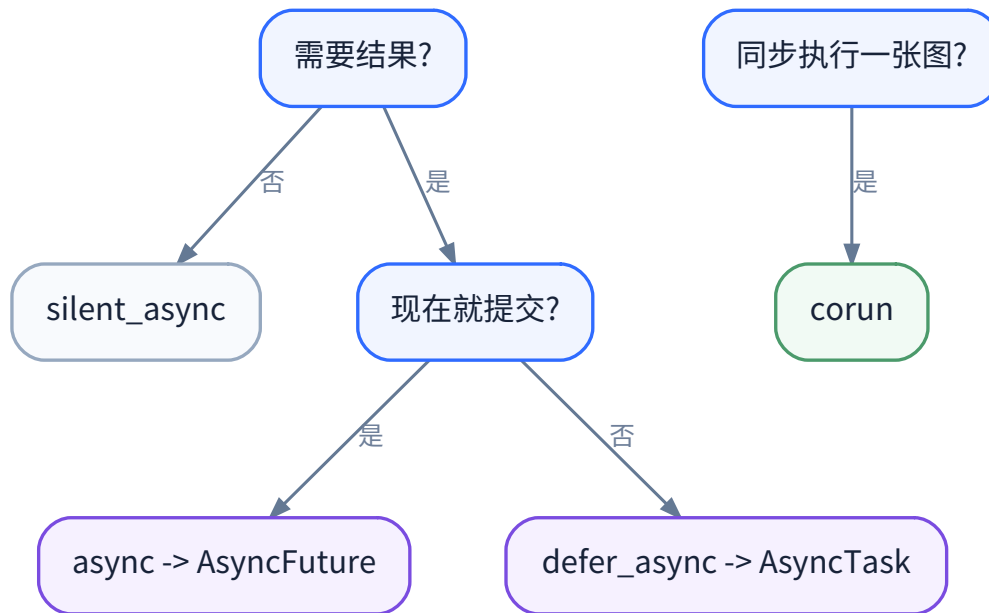


图 10 • 如何在 silent_async、async、defer_async 与 corun 之间选择

callable、Runtime、SubFlow、Graph 都可提交

```

auto a = executor.async([] { return 42; });

auto b = executor.async([](tfl::Runtime& rt) {
    return rt.worker().id();
});

auto c = executor.async([](tfl::SubFlow& sf) {
    sf.emplace([] {});
    sf.run();
    sf.wait();
    return 7;
});

tfl::Flow flow;
flow.emplace([] {});
auto d = executor.async(flow);
  
```

15. AsyncFuture: 共享完成状态与结果

`AsyncFuture<R>` 是可复制的共享句柄。

```

auto a = executor.async([] { return 42; });
auto b = a;
  
```

```
assert(a == b);
```

复制不复制任务，只增加同一个异步 Work 的强引用。

15.1 常用状态

```
future.valid();
future.running();
future.done();
future.use_count();
future.type();
future.hash_value();
```

15.2 wait 与 get 的区别

```
future.wait(); // 只等完成，不抛任务异常
future.get();  // 等完成 + 传播任务异常 + 读取结果
```

`get()` 是非消费式的，可以重复调用。

返回类型

- `R` 为值类型：返回 ResultSlot 中对象的 `const R&`；
- `R` 为左值引用：返回原始左值引用；
- `R = void`：只等待并传播异常。

因此：

```
auto future = executor.async([] { return std::string("hello"); });
const std::string& ref = future.get();
```

`ref` 的生命周期依赖底层 Work；不能释放最后一个相关句柄后继续使用它。

15.3 停止

```
future.request_stop();
if (future.stop_requested()) {
    ...
}
```

停止是协作式请求，不会强制中断正在执行的 C++ 代码。

15.4 `get()` 对不同结果类型的返回

callable 结果	<code>AsyncFuture<R>::get()</code>
<code>R = int</code>	<code>const int&</code>
<code>R = std::string</code>	<code>const std::string&</code>
<code>R = T&</code>	原始 <code>T&</code>
<code>R = void</code>	无返回值，只等待并传播异常

```
auto value = executor.async([] { return std::string{"ok"}; });
const std::string& ref = value.get();

int shared = 1;
auto reference = executor.async([&]() -> int& { return shared; });
reference.get() = 42;
```

对值结果，引用指向 Work 内部 `ResultSlot`；保存该引用时必须保证底层 Work 仍被某个 Future/Task 强引用持有。

16. AsyncTask：先配置，再启动一次

`Executor::defer_async` 返回 `AsyncTask<R>`。

```
auto task = executor.defer_async([] {
    return 42;
});

task
    .name("work")
    .acquire(resource)
    .release(resource);

task.start();
```

16.1 状态规则

- 创建后：Idle；
- 第一次成功 `start()` 后：Running；
- 完成后：Finished；
- 同一个底层任务至多成功启动一次；
- 任一共享 `AsyncTask` 副本启动后，其他副本不能再次启动。

16.2 start 的依赖规则

```
auto a = executor.async([] { return 20; });  
  
auto b = executor.defer_async([] { return 22; });  
b.start(a);
```

前驱必须已经启动或完成；Idle 前驱不能作为依赖。

空依赖句柄会被忽略：

```
tfl::AsyncFuture<void> empty;  
task.start(empty, a);
```

任务不能依赖自身。

16.3 AsyncTask 生命周期

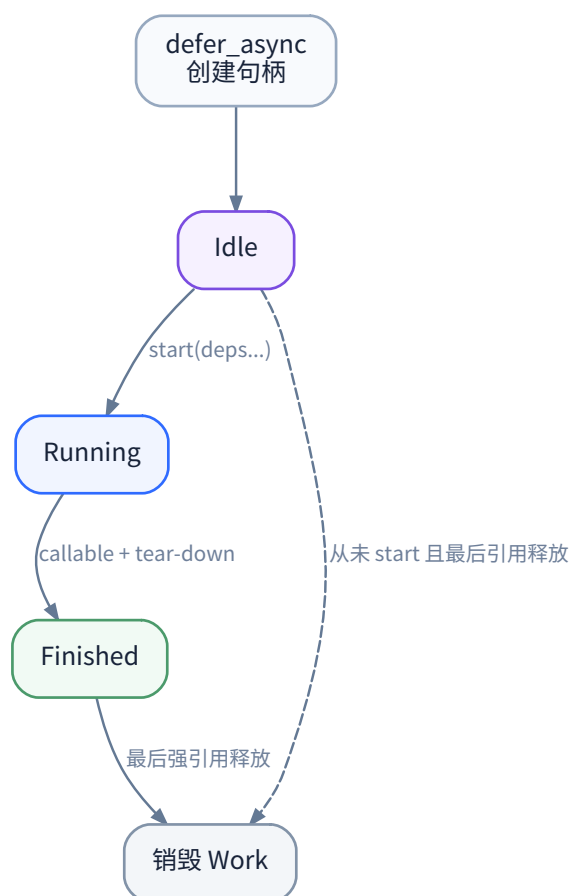


图 11 • AsyncTask 从创建到完成与销毁的生命周期

`start()` 成功后不能再次启动同一底层 Work；复制 `AsyncTask` 只是共享同一个任务，并不会得到第二次启动机会。

启动前集中配置

```
struct Trace final : tfl::TaskObserver {
    void on_before(tfl::WorkerView) override {}
    void on_after(tfl::WorkerView) override {}
};

tfl::Semaphore gpu_slots(2);
auto task = executor.defer_async([] { run_gpu_stage(); });

task.name("gpu-stage")
    .acquire(gpu_slots)
    .release(gpu_slots);

auto observer = task.register_observer<Trace>();
task.start();
task.get();
```

名称、Semaphore 和 Observer 配置都应在 `start()` 前完成，不与启动/执行并发修改。

17. 动态异步依赖到底保证什么

这是使用 Async API 时最重要的一条：

动态依赖只保证“前驱完成后，后继才具备调度条件”，并不会自动把前驱结果或前驱异常传给后继。

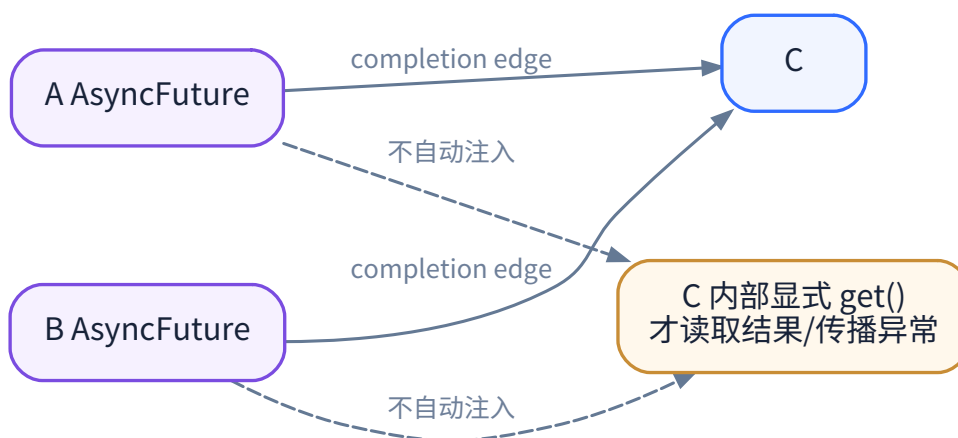


图 12 · 异步依赖只建立 completion edge，不自动传递结果或异常

正确取结果：

```

auto a = executor.async([] { return 20; });
auto b = executor.async([] { return 22; });

auto sum = executor.async([a, b] {
    return a.get() + b.get();
}, a, b);

std::cout << sum.get();    // 42

```

这里传 `a, b` 作为 `deps` 负责顺序；lambda 内的 `a.get()/b.get()` 负责读取结果并显式传播前驱异常。

17.1 为什么依赖会保留前驱生命周期

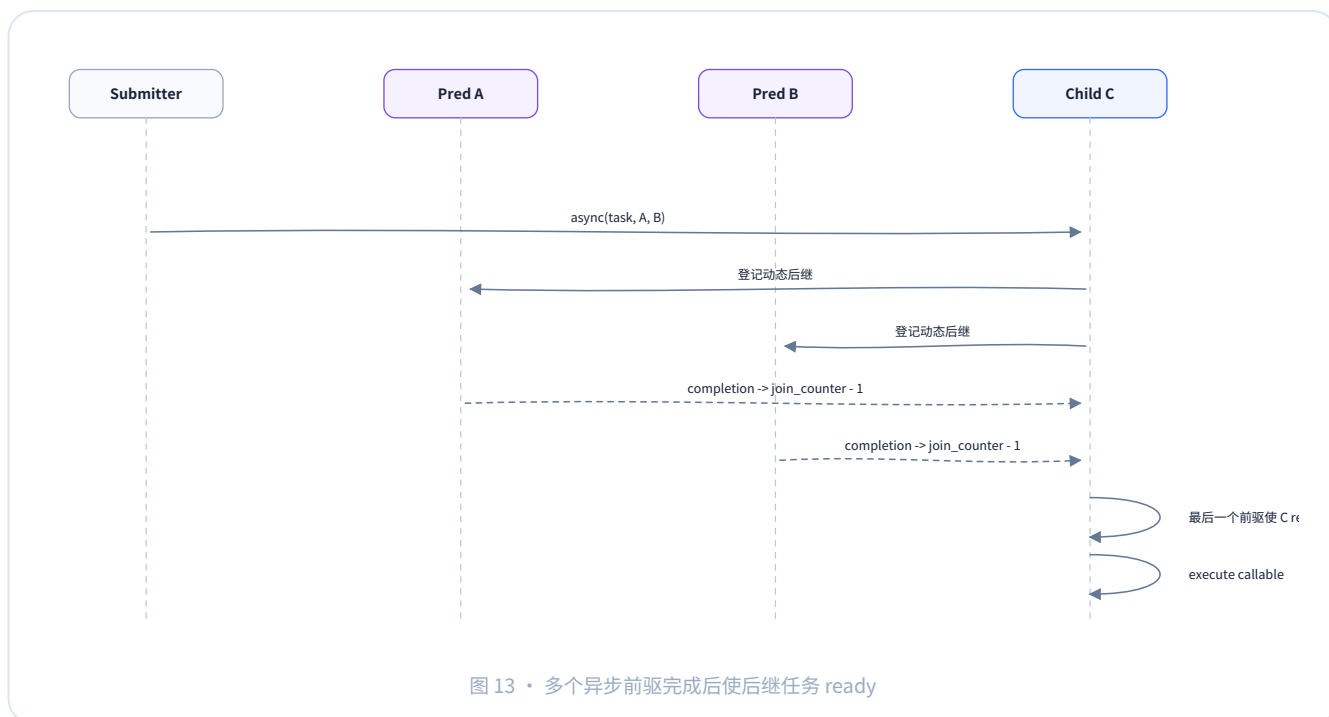
当前任务保存每个有效前驱的强引用，直到当前 Work 最终销毁。这样即使调用方先释放自己的 predecessor 句柄，动态依赖表仍不会引用已经销毁的 Work。

重复传入同一前驱属于重复条目，会按传入次数建立依赖计数/引用；不要把“重复依赖自动去重”当作接口保证。

17.2 跨 Executor

前驱与后继可以属于不同 Executor；当前后继真正 ready 时由它自己的 Executor 发布。应用仍要保证两个 Executor 的生命周期覆盖相关任务执行。

17.3 多前驱依赖的运行顺序



如果 A 在 C 登记前已经 Finished，登记阶段会直接把对应依赖视为已满足；如果 A 与登记并发完成，则通过 `Topology::Control::LOCKED` 协议串行化“完成”和“追加动态后继”。

18. AsyncTaskObject: 延迟异步任务 + 原地对象

```
struct Job {
    int value;

    explicit Job(int v) : value(v) {}

    int operator()() {
        return value * 2;
    }
};

tfl::Executor executor(4);

auto task = executor.defer_async_object<Job>(21);

std::cout << task.object().value << '\n';

task.start();

std::cout << task.get() << '\n';    // 42
```

`AsyncTaskObject<R, F>` :

- 继承 `AsyncTask<R>` ;
- `object()` 访问内部业务对象;
- `get()` 访问异步结果;
- 复制句柄共享同一个 Work 和同一个业务对象;
- `object()` 本身不等待、不加锁。

一个接口细节

继承来的链式配置函数返回 `AsyncTask<R>&` , 不会保留派生类型的对象类型信息。最清晰的写法是先保存 `AsyncTaskObject` 变量, 再单独配置和启动。

19. Executor 的 Graph 提交重载

Graph holder 同样支持三种执行形态。

19.1 async

```
auto future = executor.async(flow);
future.get();
```

定次：

```
executor.async(flow, 10).get();
```

谓词：

```
executor.async(flow, [&] {  
    return stop_condition();  
}).get();
```

可附完成回调：

```
executor.async(flow, [] {  
    std::cout << "done\n";  
}).get();
```

19.2 silent_async

```
executor.silent_async(flow);
```

没有返回句柄。引用捕获、左值 Flow 等仍必须保证生命周期覆盖执行。

19.3 defer_async

```
auto task = executor.defer_async(flow);  
task.start();  
task.get();
```

适合先配置 name、Semaphore、Observer 或动态前驱，再显式启动。

20. corun 与 wait_for_all

20.1 Executor::corun

用于“我就想同步执行这一个图”。

```
executor.corun(flow);
```

Worker 内调用时使用协作等待；外部线程调用时阻塞外部线程。

20.2 wait_for_all

```
executor.wait_for_all();
```

等待当前观察到的顶层 topology 计数归零。

它**不是**阻止其他线程继续提交的全局 **barrier**。如果另一线程正好在等待返回边界之后提交，新任务可能不属于本次观察范围。

20.3 选择

需求	建议
执行一个 Flow 并等它	<code>corun(flow)</code>
立即异步并拿结果	<code>async(...).get()</code>
Fire-and-forget 后在程序某边界等所有顶层任务	<code>wait_for_all()</code>
Worker 内等动态子任务	<code>Runtime::wait/wait_until/corun</code> 或 <code>TaskGroup::wait/corun</code>

21. Context：执行期统一上下文

`Context` 是 `Runtime`、`Branch/Jump` 等执行期对象共享的上下文基础。

可访问：

```
context.worker();
context.executor();
context.stop_requested();
context.type();
context.name();
```

它只借用当前：

- `Work`;
- `Worker`;
- `Executor`。

`Context` 和由它返回的引用不能逃逸当前任务回调，也不能跨线程保存。

这条规则同样适用于 `Runtime&`、`SubFlow&`、`Branch&`、`Jump&` 等框架注入对象。

22. Runtime：任务运行时动态派生

```
auto root = executor.async([](tfl::Runtime& rt) {
    auto a = rt.async([] { return 20; });
    auto b = rt.async([] { return 22; });

    rt.wait_until([&] {
        return a.done() && b.done();
    });

    return a.get() + b.get();
});

std::cout << root.get();
```

22.1 Runtime 的子任务归属

Runtime 派生的工作：

- 会计入当前父 Work 的完成计数；
- 异步子任务拥有独立 Topology；
- 子 Topology 以当前 Topology 为父，因此继承父级停止请求。

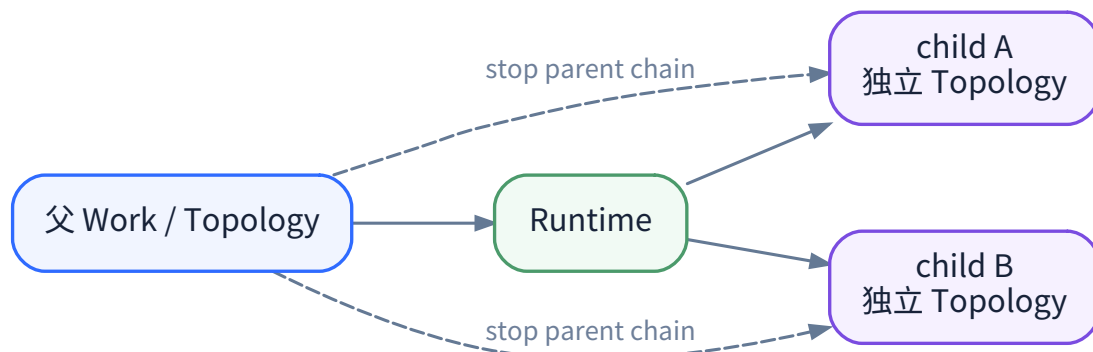


图 14 • Runtime 派生任务的父 Topology 停止继承关系

22.2 run(graph)

```
rt.run(flow);
```

把子图 source 节点挂到当前 Work，然后立即返回。

随后：

```
rt.wait();
```

等待当前 Runtime 父 Work 已挂接的动态子工作。

22.3 corun(graph)

```
rt.corun(flow);
```

建立一个独立局部 AnchorWork，只等待这次子图，并在该局部边界重抛异常。

当当前 Runtime 还挂着其他独立子任务，而你只想等某一个图时，`corun` 比 “run + wait 所有孩子” 更精确。

22.4 wait_until

```
rt.wait_until([&] {
    return condition.load(std::memory_order_acquire);
});
```

等待期间当前 Worker 会继续执行本地/窃取到的 ready Work。谓词可能被多次调用，应短小、无阻塞并使用正确同步。

22.5 Runtime 常用接口表

接口	行为	是否等待
<code>silent_async(task)</code>	派生 fire-and-forget 子任务	否
<code>async(task, deps...)</code>	派生带 Future 的子任务	否
<code>run(graph)</code>	把图挂到当前 Work	否
<code>corun(graph)</code>	在独立锚点下执行图	是，协作等待
<code>wait()</code>	等当前 Work 已挂接子任务	是，协作等待
<code>wait_until(pred)</code>	执行调度直到谓词成立	是，协作等待

Runtime fan-out + 汇总

```
auto total = executor.async([&](tfl::Runtime& rt) {
    auto a = rt.async([&] { return 20; });
    auto b = rt.async([&] { return 22; });

    rt.wait_until([&] { return a.done() && b.done(); });
    return a.get() + b.get();
});
```

```
});  
  
assert(total.get() == 42);
```

run 与 corun 的范围差异

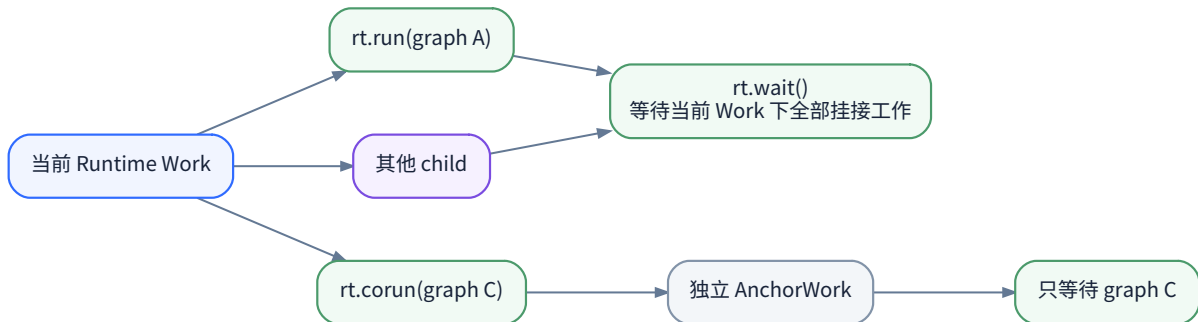


图 15 • Runtime::run/wait 与 Runtime::corun 的等待范围差异

23. SubFlow：执行期间动态构图

```
auto parent = flow.emplace([](tfl::SubFlow& sf) {  
    auto a = sf.emplace([] {});  
    auto b = sf.emplace([] {});  
    a.precede(b);  
  
    sf.run();  
    sf.wait();  
});
```

SubFlow:

- 同时是 `FlowBuilder` 和 `Context`；
- 每次进入该 SubFlow 节点会清空并重建其动态图；
- 只构图不调用 `run()`，不会自动执行；
- 上一轮创建的 Task 句柄在下一轮重建后失效。

23.1 run 与 wait

```
sf.run(); // 发布 source，立即返回  
sf.wait(); // 协作等待当前父节点挂接工作
```

生命周期陷阱

如果动态子任务引用父 callable 的局部变量，必须在这些局部变量离开作用域前完成等待：

```

flow.emplace([](tfl::SubFlow& sf) {
    int local = 42;

    sf.emplace([&] {
        use(local);
    });

    sf.run();
    sf.wait(); // 必须在 local 析构前
});

```

23.3 动态数量节点示例

```

auto parent = executor.async([&](tfl::SubFlow& sf) {
    std::vector<tfl::Task> tasks;
    tasks.reserve(inputs.size());

    for (std::size_t i = 0; i < inputs.size(); ++i) {
        tasks.emplace_back(sf.emplace([&, i] {
            outputs[i] = transform(inputs[i]);
        }));
    }

    sf.run();
    sf.wait();
    return outputs.size();
});

```

SubFlow 特别适合“每次调用节点数量/依赖关系由本次输入决定”的场景。下一轮同一 SubFlow callable 执行时，内部动态图会重新构建，上一轮取得的 `Task` 句柄不能继续保存使用。

24. TaskGroup：局部分组、停止与异常边界

```

auto root = executor.async([](tfl::Runtime& rt) {
    tfl::TaskGroup group(rt);

    auto a = group.async([] { return 20; });
    auto b = group.async([] { return 22; });

    group.wait();

    return a.get() + b.get();
});

```

24.1 TaskGroup 当前语义

- 构造参数是 `Context&`；

- 内部建立独立 `AnchorWork` ；
- 组内工作都挂在这个 `Anchor` 下；
- 组内异步任务继承当前 `Context` 的停止域；
- 析构函数 `noexcept` ，会协作等待未完成组任务；
- 析构不会重新抛出组内异常；
- 要在当前作用域观察异常，必须显式调用 `group.wait()` 。

24.2 `run` 与 `corun`

```
group.run(flow);    // 提交到组，立即返回
group.wait();       // 等组内全部工作

group.corun(flow);  // 只等这次图
```

24.3 独立于父停止域的任务

如果业务上需要独立顶层任务，不要通过当前 `group/runtime` 派生，而是显式使用：

```
auto independent = rt.executor().async([] {
    ...
});
```

它是 `Executor` 顶层异步任务，不以当前 `Runtime` 的 `Topology` 为父。

24.4 `TaskGroup` 的局部批处理

```
auto parent = executor.async([](tfl::Runtime& rt) {
    tfl::TaskGroup group(rt);

    auto a = group.async([] { return 20; });
    auto b = group.async([] { return 22; });

    group.wait();
    return a.get() + b.get();
});
```

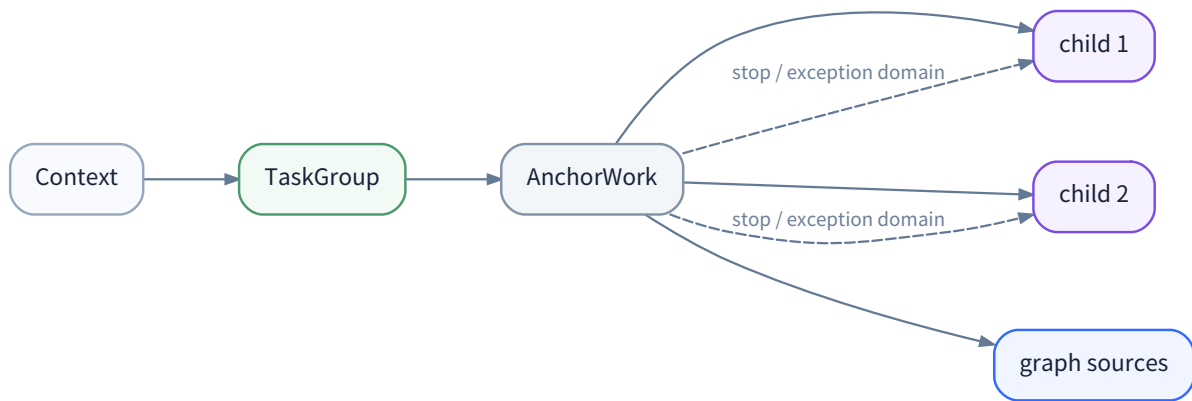


图 16 • TaskGroup 通过 AnchorWork 聚合完成、停止与异常边界

TaskGroup 析构会确保组内任务完成，但析构是 `noexcept`；要在当前作用域处理组内异常，显式调用 `group.wait()`。

25. 为什么 Worker 内要协作等待

最典型的线程池死锁模式：

```

只有 1 个 Worker
父任务正在 Worker 0 上执行
父任务创建 child
父任务调用普通阻塞等待
Worker 0 被堵住
child 虽然 ready, 却没有 Worker 执行
  
```

TaskflowLite 的 `Runtime::wait`、`wait_until`、`corun`、`TaskGroup::wait/corun` 会：

1. 先检查等待条件；
2. 从当前本地队列拿工作；
3. 窃取其他队列；
4. 执行拿到的 Work；
5. 再检查目标条件。

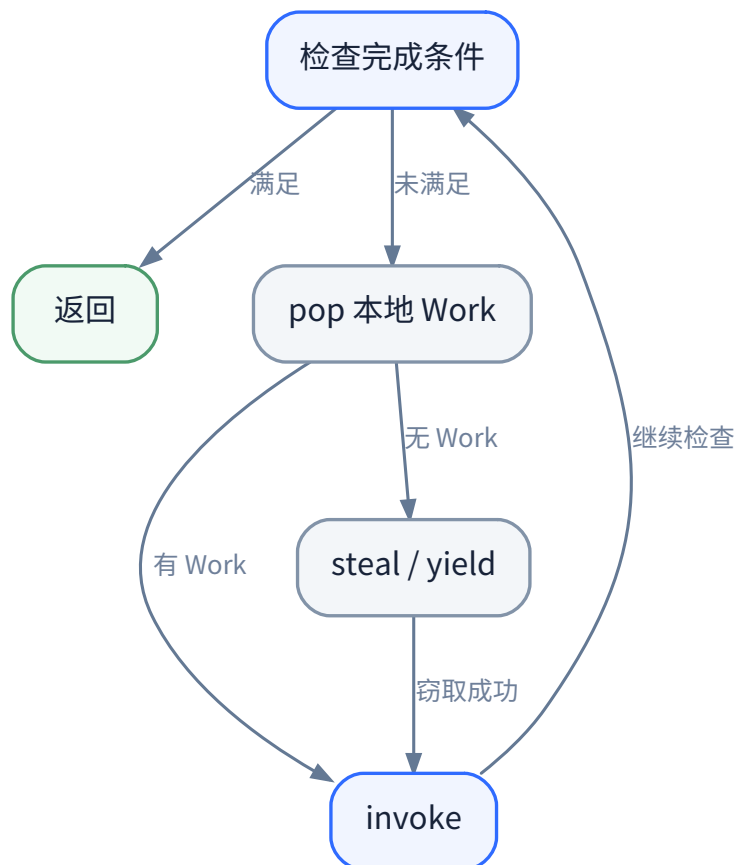


图 17 • 协作等待循环：等待期间 Worker 继续推进其他 ready Work

因此 Worker 在“等待”时仍然是调度系统的一部分。

普通 `AsyncFuture::wait/get` 是阻塞线程的接口；Worker 内优先选择协作式等待。

26. 协作式停止

停止请求是状态，不是抢占中断。

```

auto future = executor.async([](tfl::Runtime& rt) {
    while (!rt.stop_requested()) {
        do_one_piece();
    }
});

future.request_stop();
future.get();
  
```

26.1 停止沿父 Topology 链继承

Runtime、TaskGroup 派生任务会检查当前 Topology 及祖先 Topology。

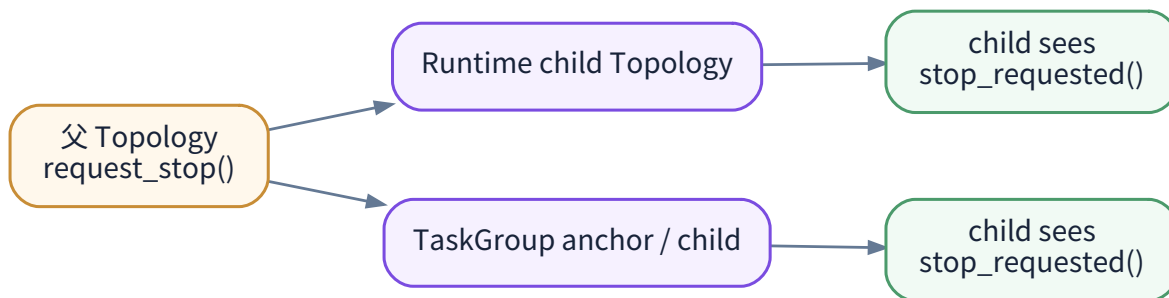


图 18 • 停止请求沿 Topology 父链向下传播

顶层 `executor.async` 创建独立停止域。

26.2 停止不会自动做的事

停止请求不会：

- 中断正在执行的系统调用；
- 杀线程；
- 自动撤销业务数据；
- 自动回滚已经产生的副作用。

可取消任务必须把工作切成可检查的小块。

27. 异常传播模型

27.1 普通任务异常

异常支持开启时，用户 callable 抛出的异常进入 Work 的统一归档/传播流程，不会直接从 Worker 线程函数逃出。

顶层常见观察边界：

```

try {
    executor.corun(flow);
} catch (const std::exception& e) {
    ...
}
  
```

或：

```
auto future = executor.async(...);

try {
    future.get();
} catch (...) {
    ...
}
```

`wait()` 只负责完成同步，不负责传播异步任务异常。

27.2 Observer 异常

`TaskObserver::on_before/on_after` 当前允许抛异常。框架把它们和 callable 异常一样纳入当前 Work 的异常流程。

不要把 Observer 当作“永远 noexcept 的日志接口”；观察器出错会影响对应任务执行结果。

27.3 TaskGroup

TaskGroup 析构必须 `noexcept`，因此：

```
{
    tfl::TaskGroup group(rt);
    group.async(...);
} // 会等，但不能靠析构接异常
```

需要捕获：

```
try {
    group.wait();
} catch (...) {
    ...
}
```

27.4 异常从哪里观察



图 19 • 任务异常的归档、局部锚点与重新抛出路径

常见同步边界：

- `AsyncFuture::get()`；
- `Executor::corun()`；
- `Runtime::corun()`；
- `TaskGroup::wait()`；
- `Runtime/SubFlow::wait()` 在对应显式异常锚点存在时。

只调用 `AsyncFuture::wait()` 不会重抛任务异常。

28. ScopedExceptionAnchor：显式局部异常边界

默认情况下，动态子任务异常可以沿父链向外传播。

如果希望在当前 `Runtime/SubFlow` 作用域**截断向上传播并由当前等待点接住**：

```
#include <taskflowlite/core/scoped_exception_anchor.hpp>

flow.emplace([](tfl::SubFlow& sf) {
    tfl::ScopedExceptionAnchor anchor{sf};

    sf.emplace([] {
        throw std::runtime_error("child error");
    });

    sf.run();

    try {
        sf.wait();
    } catch (const std::runtime_error&) {
        // 在当前动态作用域处理
    }
});
```

必须遵守

- 在要拦截的子任务**启动前**创建；
- 保持到子任务全部完成、对应 `wait` 返回；
- 同一 `Work` 上多个 `anchor` 必须严格嵌套；
- 不能跨线程；
- `anchor` 本身借用 `Context` 当前 `Work`。

29. Semaphore：任务级资源配额

TaskflowLite 的 Semaphore 不是让 Worker 阻塞在 `acquire()` 上，而是把暂时拿不到资源的 Work 挂入 waiter 链。

```
tfl::Semaphore gpu_slots(2, "gpu");

auto task = flow.emplace([] {
    use_gpu();
});

task.acquire(gpu_slots)
    .release(gpu_slots);
```

也支持计数：

```
task.acquire(gpu_slots, 2);
task.release(gpu_slots, 2);
```

29.1 多 Semaphore 是 all-or-nothing

当一个 Work 同时需要多个资源：

```
task.acquire(cpu_slot);
task.acquire(gpu_slot);
```

执行前会按固定全局顺序锁定目标 Semaphore：

1. 检查所有请求能否同时满足；
2. 全部满足：一次性扣减；
3. 任意一个不满足：一个都不扣；
4. 把 Work 放进缺资源的等待链；
5. 释放资源后，waiter 被重新发布；
6. 被唤醒 Work 重新竞争它的全部资源。



图 20 · 多 Semaphore all-or-nothing 获取协议

这避免“先拿到 A、拿不到 B、自己占着 A 等 B”的资源死锁模式。

29.2 reset

```
tfl::Semaphore sem(4);

bool ok = sem.reset(8);
bool ok2 = sem.reset(8, 3);
```

- 有等待任务时返回 `false` ；
- 没 waiter 时更新容量；
- `current_value > max_value` 会裁剪为 `max_value` ；
- 调用方必须保证没有尚未归还的占用配额。

29.3 生命周期

Task 只保存 Semaphore 地址，不拥有它。所有可能访问该资源的任务完成之前，Semaphore 必须继续存活。

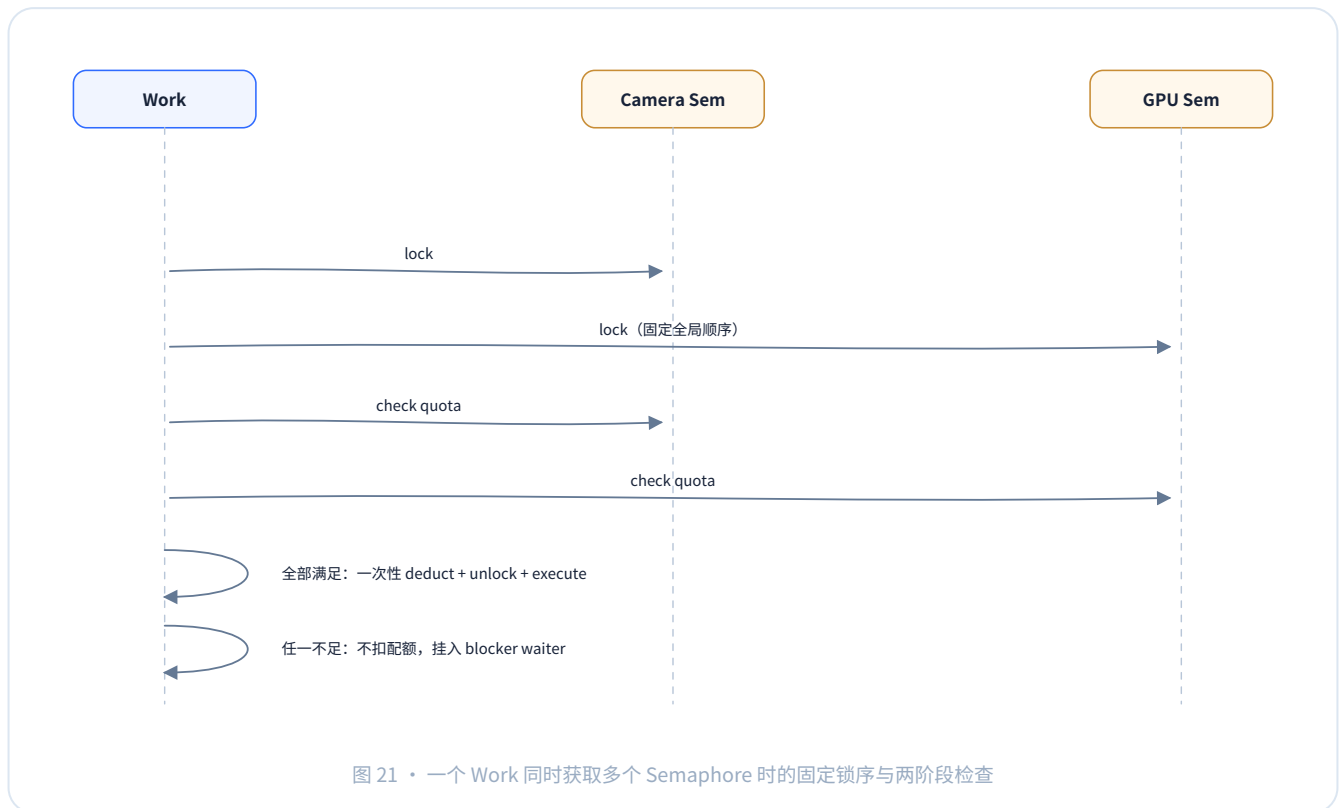
29.4 多资源获取示例

```
tfl::Semaphore camera(1, "camera");
tfl::Semaphore gpu(2, "gpu");

auto task = flow.emplace([] {
    process_frame();
});

task.acquire(camera)
    .acquire(gpu)
    .release(gpu)
    .release(camera);
```

内部不会按调用顺序一个个“拿锁”。Work 会把 acquire 请求按 Semaphore 地址建立固定顺序，在统一持锁后先检查全部配额：



`reset()` 只适合资源静止期配置；当前存在 waiter 时返回 `false` 并保持原状态。

30. TaskObserver: 任务执行观察

```

class TraceObserver : public tfl::TaskObserver {
public:
    void on_before(tfl::WorkerView worker) override {
        std::cout << "before on worker " << worker.id() << '\n';
    }

    void on_after(tfl::WorkerView worker) override {
        std::cout << "after on worker " << worker.id() << '\n';
    }
};
  
```

注册:

```
auto observer = task.register_observer<TraceObserver>();
```

注销:

```
task.unregister_observer(observer);
```

异步延迟任务也支持 Observer 配置。

并发要求

同一个 Observer 实例可能被多个 Worker 同时回调，内部共享状态必须自行同步。

性能建议

Observer 位于实际任务调用路径，不要在热任务中做重锁、大量格式化或同步 I/O。高频 tracing 更适合写线程本地缓冲，最后批量输出。

31. WorkerHandler：线程生命周期钩子

```
class Handler : public tfl::WorkerHandler {
public:
    void on_start(tfl::Worker& worker) noexcept override {
        // 设置线程名、亲和性、线程局部资源等
    }

    void on_stop(tfl::Worker& worker) noexcept override {
        // 清理线程局部资源
    }
};

Handler handler;
tfl::Executor executor(handler, 4);
```

规则：

- handler 由应用拥有；
- `on_start/on_stop` 在目标 Worker 自己的 OS 线程调用；
- 同一 handler 会被多个 Worker 并发调用；
- 两个 hook 都是 `noexcept`，派生实现不得抛异常；
- 不要 join、移动或替换 `Worker::thread()`。

Worker 可查询：

```
worker.id();
worker.queue_size();
worker.queue_capacity();
worker.thread().get_id();
```

这些队列数据只是瞬时快照。

30.3 一个可计时的 TaskObserver

```
struct TimingObserver final : tfl::TaskObserver {
    std::atomic<std::uint64_t> starts{0};
    std::atomic<std::uint64_t> finishes{0};

    void on_before(tfl::WorkerView) override {
        starts.fetch_add(1, std::memory_order_relaxed);
    }

    void on_after(tfl::WorkerView) override {
        finishes.fetch_add(1, std::memory_order_relaxed);
    }
};

auto observer = task.register_observer<TimingObserver>();
executor.corun(flow);
assert(observer->starts == observer->finishes);
```

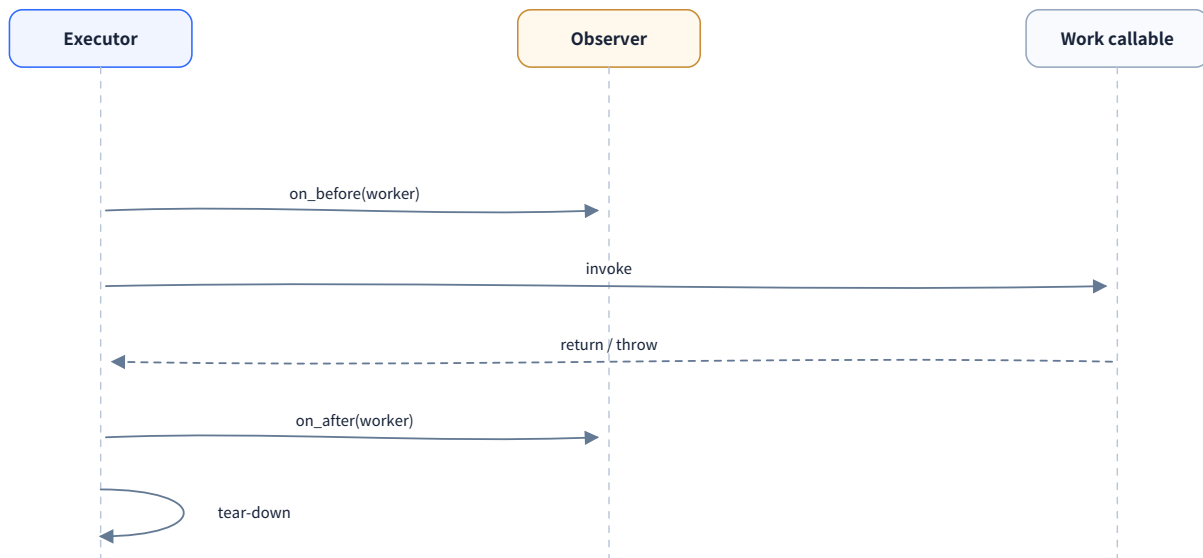


图 22 • TaskObserver 在 callable 前后执行，异常统一进入 Work 归档路径

Observer 可能被多个 Worker 并发调用；观察者内部共享状态需要自行同步。Observer 回调抛出的异常也进入当前 Work 的统一异常归档路径。

31.3 WorkerHandler 的典型用途

```
struct Handler final : tfl::WorkerHandler {
    void on_start(tfl::Worker& worker) noexcept override {
        // 设置线程名、亲和性、TLS 等
        register_worker(worker.id());
    }

    void on_stop(tfl::Worker& worker) noexcept override {
        unregister_worker(worker.id());
    }
};
```

```

    }
};

Handler handler;
tfl::Executor executor(handler, 4);

```

它观察的是 **Worker 线程生命周期**，不是每个任务；两个 hook 都运行在对应 Worker 自己的 OS 线程上，并且必须 `noexcept`。

32. D2 图导出与命名

```

tfl::Flow flow{"pipeline"};

auto a = flow.emplace([] {} ).name("decode");
auto b = flow.emplace([] {} ).name("infer");
a.precede(b);

std::cout << flow.dump(tfl::Direction::Right);

```

布局方向：

```

tfl::Direction::Down
tfl::Direction::Right
tfl::Direction::Up
tfl::Direction::Left

```

Task、AsyncFuture 也有 `dump` 接口。

D2 输出描述的是**结构与节点配置**，不是执行时序 trace。要观察运行历史使用 Observer 或外部 tracing。

32.3 导出到文件

```

tfl::Flow flow("vision-pipeline");
// ... emplace / precede ...

std::ofstream out("flow.d2");
flow.dump(out, tfl::Direction::Right);

```

也可以直接：

```

std::cout << flow.dump(tfl::Direction::Down);

```

D2 输出描述的是**当前图结构**；它不是运行时 trace。要分析实际 Worker 执行顺序，应结合 `TaskObserver` 或外部 tracing。



图 50 • dump 输出结构描述；真实执行历史应由 Observer / tracing 记录

33. 并行模式一：fan-out / fan-in

最常见模式：

```

tfl::Flow flow;
std::array<int, 4> data{};
int sum = 0;

auto a = flow.emplace([&] { data[0] = 10; });
auto b = flow.emplace([&] { data[1] = 20; });
auto c = flow.emplace([&] { data[2] = 30; });
auto d = flow.emplace([&] { data[3] = 40; });

auto reduce = flow.emplace([&] {
    for (int v : data) {
        sum += v;
    }
});

a.precede(reduce);
b.precede(reduce);
c.precede(reduce);
d.precede(reduce);

executor.corun(flow);
  
```

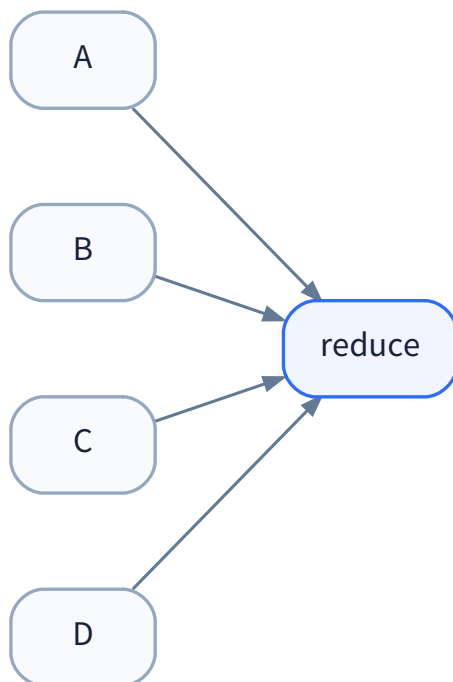


图 23 • 并行 fan-out 后汇聚到归约节点

这是当前 core 实现 `parallel reduce` 的基础积木：分块任务并行产生部分结果，最后一个汇聚任务归并。

34. 并行模式二：索引分块

当前没有正式 `for_each_index` API，可以显式分块：

```
template <typename F>
void parallel_for_index(
    tfl::Flow& flow,
    std::size_t first,
    std::size_t last,
    std::size_t chunks,
    F&& fn) {

    const std::size_t n = last - first;
    const std::size_t block = (n + chunks - 1) / chunks;

    for (std::size_t c = 0; c < chunks; ++c) {
        const std::size_t begin = first + c * block;
        const std::size_t end = (std::min)(last, begin + block);

        if (begin >= end) {
            break;
        }

        flow.emplace([=, &fn] {
```

```

        for (std::size_t i = begin; i < end; ++i) {
            fn(i);
        }
    });
}
}

```

实际工程还应根据任务粒度、缓存局部性和 Worker 数选择 chunk 数量，不要把“一个元素一个 Task”当成默认最优策略。

35. 并行模式三：Pipeline、生产者/消费者和状态机

35.1 Pipeline

用阶段间依赖表达：

```
decode -> preprocess -> infer -> postprocess -> write
```

当处理多帧时，可以把每帧作为独立状态，跨帧允许不同阶段重叠，但共享设备可用 Semaphore 限制。

35.2 Producer / Consumer

动态数量生产适合 Runtime：

```

executor.async([](tfl::Runtime& rt) {
    for (...) {
        rt.silent_async([item = ...] {
            consume(item);
        });
    }

    rt.wait();
}).get();

```

如果需要结果集合，使用 `rt.async` 保存 futures，再 `wait_until` / `wait` 后读取。

35.3 状态机 / Retry

Branch 适合“选择正常路径”，Jump 适合“强制重新激活某个状态”。

设计循环时建议显式维护：

- 最大尝试次数；

- 停止请求检查；
- 每次重试的业务状态；
- 错误退出路径。

避免构造没有任何终止条件的 Jump 环。

35.3 一个完整的三阶段 Pipeline

```
tfl::Flow flow;
std::array<Item, 4> items;
std::array<Result, 4> results;

std::array<tfl::Task, 4> decode;
std::array<tfl::Task, 4> infer;

for (std::size_t i = 0; i < items.size(); ++i) {
    decode[i] = flow.emplace([&, i] { decode_item(items[i]); });
    infer[i] = flow.emplace([&, i] { results[i] = infer_item(items[i]); });
    decode[i].precede(infer[i]);
}

auto write = flow.emplace([&] { write_batch(results); });
for (auto task : infer) task.precede(write);

executor.corun(flow);
```

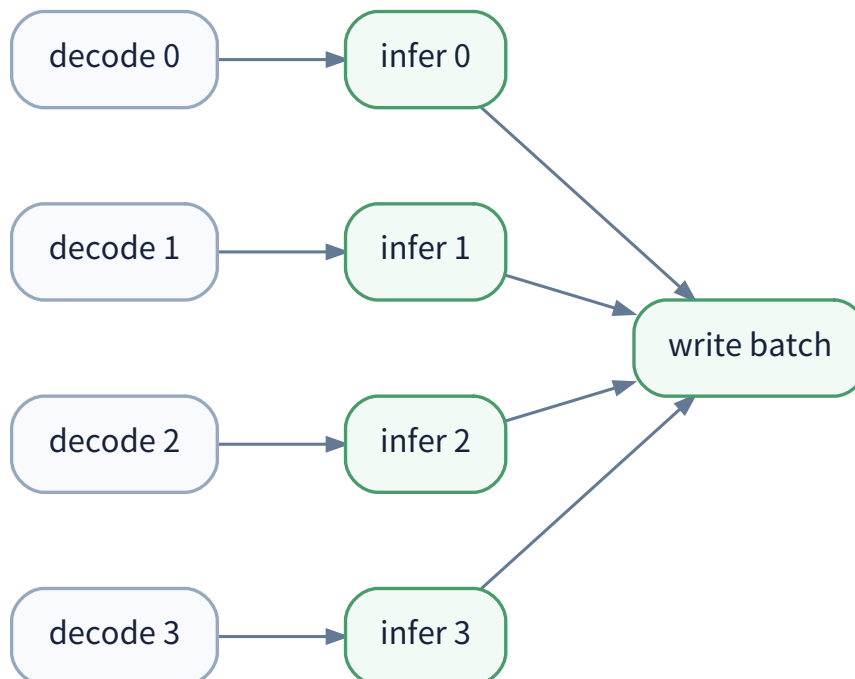


图 24 • Pipeline 中多个处理支路并行后汇聚写出

这种写法适合阶段固定、每个元素工作独立的 CPU/GPU 前后处理。若每轮节点数量动态变化，把创建逻辑放到 `SubFlow` 或 `Runtime`。

35.4 Map-Reduce 模式

```
std::array<int, 4> partial{};
std::array<tfl::Task, 4> map;

for (std::size_t i = 0; i < map.size(); ++i) {
    map[i] = flow.emplace([&, i] { partial[i] = compute_chunk(i); });
}

auto reduce = flow.emplace([&] {
    total = std::accumulate(partial.begin(), partial.end(), 0);
});

for (auto task : map) task.precede(reduce);
```

注意：TaskflowLite 当前没有公开的 `parallel_reduce` API；这里是用现有任务图表达同样的依赖模式。

36. Executor 视角：一次图执行发生什么

这一节不是要求普通用户依赖私有函数，而是帮助排查卡住、重复执行、队列与计数问题。

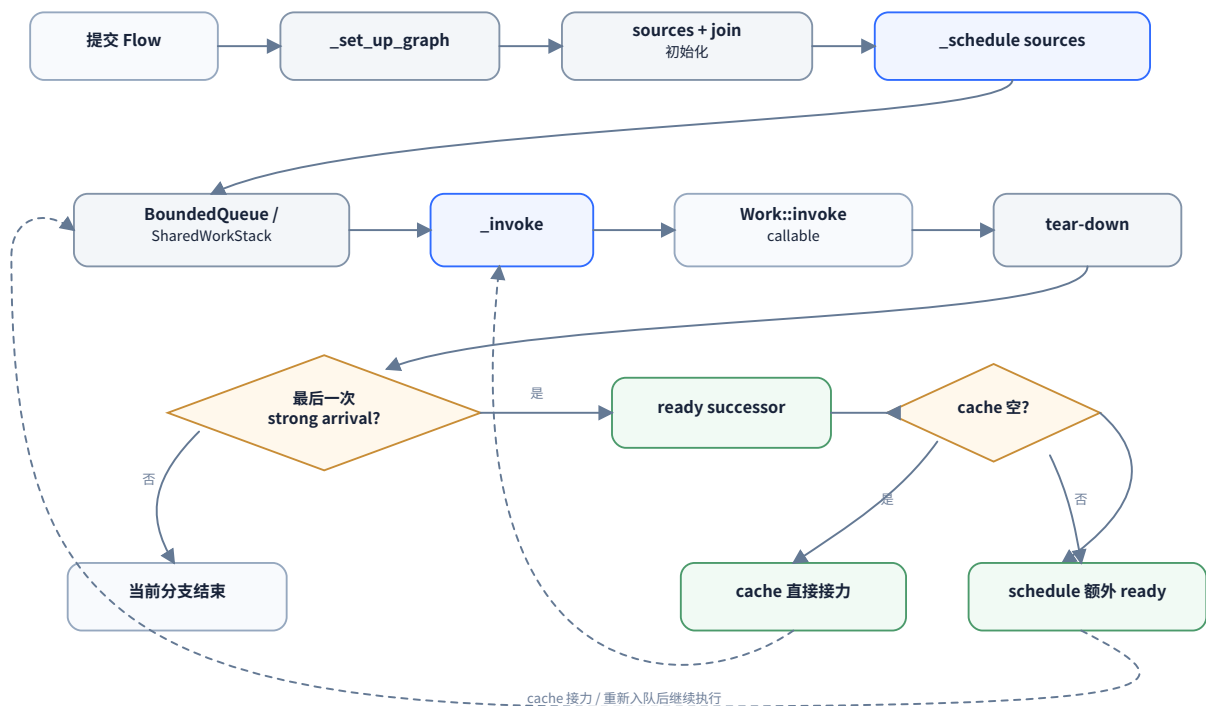


图 25 • 静态图从 setup、schedule、invoke 到 tear-down 的完整执行链

36.1 _set_up_graph

每轮执行前会：

- 给节点重新绑定 parent / topology；
- 清理上一轮运行时异常状态；
- 计算静态 strong join weight；
- 初始化非 source 的运行期 join_counter；
- 将物理零入度 source 聚集到图存储前段。

“source” 依据的是物理前驱数，不是只看 strong join weight。

36.2 cache 接力

完成节点若只有一个合适的 ready 后继，可以直接让当前 Worker 在同一调用栈继续执行，减少：

- queue push；
- queue pop；
- Notifier 唤醒；
- 再次 steal。

这也是小任务图的重要性能路径。

36.4 corun 在外部线程和 Worker 内部的差异

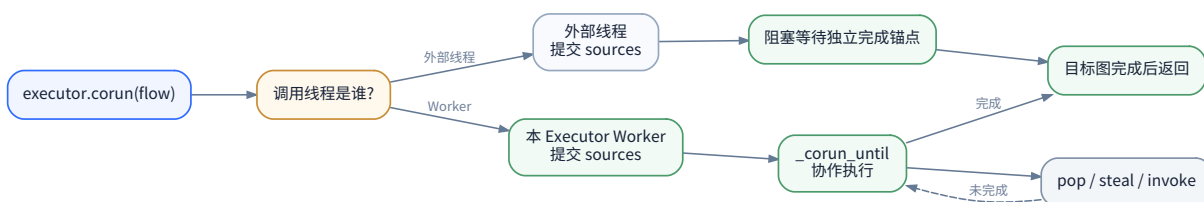


图 26 • Executor::corun 在外部线程与 Worker 内部的两种等待路径

这也是为什么在单 Worker 场景中，Worker 内部同步跑子图应优先使用 corun / Runtime 协作等待，而不是在同一个 Worker 上直接阻塞普通 Future。

37. 本地队列与 SharedWorkStack

37.1 Worker 本地 BoundedQueue

当前 Worker 每个拥有固定容量 work-stealing 队列：

```
Owner:
  push / pop -> LIFO

Stealer:
  steal -> FIFO
```

Owner 的 LIFO 有利于局部性；Stealer 从另一端拿较老工作，减少与 Owner 竞争同一热点位置。

37.2 队列满时

本地队列不是自动无限扩容。

当本地 push 溢出时，Executor 把溢出工作发布到共享调度分片。

37.3 SharedWorkStack

当前版本已经不再使用旧的共享 UnboundedQueue 路线，而是用多个 `SharedWorkStack`：

- ready Work 复用自身 intrusive `m_next`；
- 外部提交和本地溢出进入共享分片；
- 单 Work 按地址哈希选择分片；
- 批量 Work 尽量均匀分散；
- Worker 窃取时把本地 Worker 队列和共享分片都视为 victim 候选。

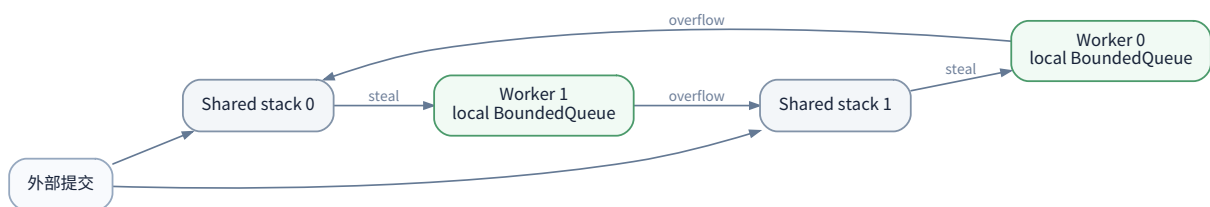


图 27 • Worker 本地 BoundedQueue 与分片 SharedWorkStack 的协作关系

38. Work stealing 与 Notifier

Worker 没有本地任务时并不是立刻睡眠。

典型路径：

1. 随机/历史 victim 探测；
2. 快速连续 steal；
3. 超过阈值后 `yield` 退避；
4. 仍无工作，进入 Notifier 两阶段协议；

5. prepare wait;
6. 再检查是否已有工作;
7. 有工作则 cancel wait;
8. 仍无工作才 commit wait。

38.1 为什么必须两阶段

危险窗口：

```
Worker: 检查“没任务” -----> 真正睡眠
Producer:          push + notify
```

如果 notify 恰好发生在 Worker “准备睡但还没登记为 waiter” 的窗口，就可能 lost wake-up。

Notifier 使用：

```
prepare -> double-check -> commit/cancel
```

关闭这个窗口。

38.2 当前 Notifier 内部

当前实现把：

- epoch;
- prewaiter count;
- waiter stack top;

打包到一个 64-bit 原子状态中，并给每个 Worker 一个 Waiter 三态：

```
NotSignaled -> Waiting -> Signaled
```

这属于调度器内部协议，应用不应直接依赖其位布局。

39. Work、Graph 与边表布局

Graph 物理拥有 Work；Work 同时保存执行 Payload、边、计数、父关系和运行态。

当前静态图边表的重要布局：

```
m_edges
[ successors ... ][ predecessors ... ]
```

```

      ^           ^
      0         m_num_successors

```

`m_num_successors` 是分界点。

这样 Work 可以用一个紧凑容器保存双向关系，同时快速拿到后继前缀。

39.1 两种 parent 不要混淆

Work parent

用于完成计数/动态层级：

```

parent Work
└─ child Work

```

Topology parent

用于停止状态和某些生命周期语义继承：

```

parent Topology
└─ child Topology

```

同一个动态子任务可以有独立 Topology，但它的 Topology 仍指向父停止域。

40. join_counter: 依赖到达与 parent slot 守恒

这是当前 core 最关键的不变量之一。

40.1 静态 strong dependency

某节点有 N 个 strong predecessor：

```

初始 join_counter = N

pred 1 完成: fetch_sub -> N-1
pred 2 完成: fetch_sub -> N-2
...
最后一个 pred: 1 -> 0
最后一个到达者取得调度权

```

当前实现统一让 strong predecessor 都通过 `fetch_sub` 参与，不再把单前驱节点特殊处理成非原子 ready 路径。

40.2 执行完成后恢复静态 weight

为了支持受控循环/重复激活，节点完成时先把自己的静态 join weight `fetch_add` 回去，再传播后继。

为什么不能简单 `store(weight)`：

```
当前节点还在 tear-down  
下一轮前驱可能已经提前到达并递减 counter  
store 会覆盖这个已经发生的到达  
fetch_add 会保留它
```

40.3 parent slot

一个已调度的静态 Work 通常占父级一个完成槽。

完成时：

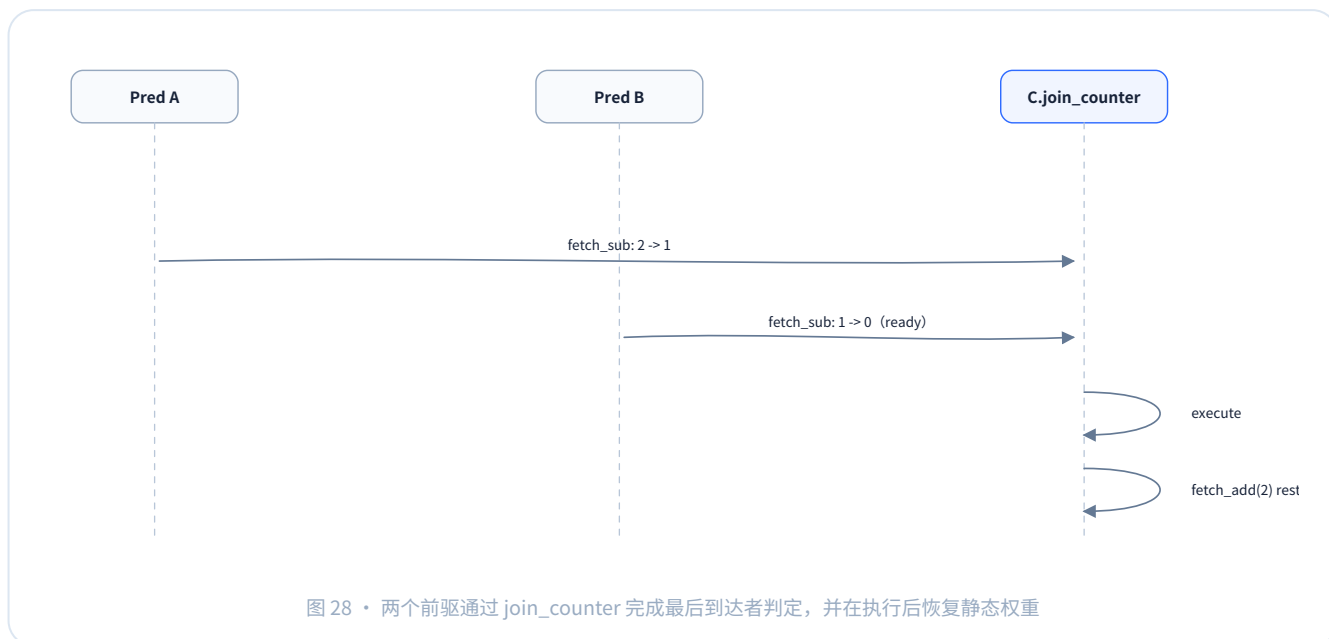
- 若有一个 ready 后继可 cache 接力，它继承当前 slot；
- 更多 ready 后继要额外给 parent 增加 slot；
- 没有 ready 后继，则归还当前 slot。

这个“slot 守恒”让 Runtime / Module / SubFlow 的完成判断不必为每条链反复创建额外同步对象。

40.4 两前驱汇聚的计数时间线

假设 `A -> C`、`B -> C`，则 C 的静态 `join_weight = 2`：

```
初始: C.join_counter = 2  
A 完成: fetch_sub(1) 旧值 2 -> 1, 不调度 C  
B 完成: fetch_sub(1) 旧值 1 -> 0, B 取得 C 本轮执行权  
C 开始执行  
C 完成: fetch_add(join_weight=2), 恢复到下一轮需要的状态
```



恢复使用 `fetch_add` 而不是 `store`，是为了保留循环控制流中“下一轮前驱已经提前到达”的递减进度。

41. Branch、Jump 在 tear-down 阶段的区别

普通任务

向所有后继传播 strong arrival。

Branch

只向选中目标传播正常 strong arrival。

MultiBranch

只向选中的多个目标传播 strong arrival。

Jump

把目标运行期 `join_counter` 清零，绕过普通 strong barrier，直接激活。

MultiJump

对多个目标执行强制激活。

这就是为什么 Branch 适合 DAG 条件路径，而 Jump 能构造受控循环。

42. 异步动态依赖的内部协议

动态 AsyncTask 不能像静态图那样提前拥有固定 predecessor/successor 邻接表，因为前驱可能：

- 已经完成；
- 正在完成；
- 正在被另一个线程连接新后继。

当前协议用 `Topology::Control` 的状态和 `LOCKED` 位协调：

```
Idle -> Running -> Finished
      +
      LOCKED bit
```

连接后继时：

1. 读取前驱状态；
2. 若已经 Finished，直接把依赖视为满足；
3. 否则取得前驱控制锁；
4. 再确认状态；
5. 把后继追加到前驱动态 successor 表；
6. 解锁。

前驱完成时只有在没有连接线程持锁时才能发布 Finished；Finished 发布后 successor 前缀冻结，然后逐个递减后继 `join_counter`。

提交保护计数

新后继注册前驱期间会多保留一个保护计数，避免某个前驱过快完成，把后继在依赖表尚未完全登记时提前调度。

这是异步“连接”和“完成”并发安全的核心。

43. 内存、对象和结果存储

43.1 Work 对象池

`TFL_ENABLE_TASK_POOL` 默认开启，Work 创建/销毁可以走共享对象池以降低频繁 new/delete 成本。

关闭：

```
#define TFL_ENABLE_TASK_POOL 0
#include <taskflowlite/taskflowlite.hpp>
```

所有翻译单元的布局/行为相关宏应保持一致。

43.2 SmallVector

Work 的边、动态目标、观察数据等热点小容器会使用 SmallVector，目标是：

- 常见小规模不分配；
- 需要时再扩容；
- 减少指针间接和 allocator 压力。

43.3 ResultSlot

异步结果跟随 Work 生命周期保存。

因此 `AsyncFuture::get()` 对值类型返回 const 引用，而不是每次复制结果。

43.4 callable / object 原地存储

TaskObject 和 AsyncTaskObject 让业务类型直接构造在 Work 的 Invoker/Payload 存储路径中，不要求先创建临时 T 再移动进去。

43.5 Async Work 生命周期图

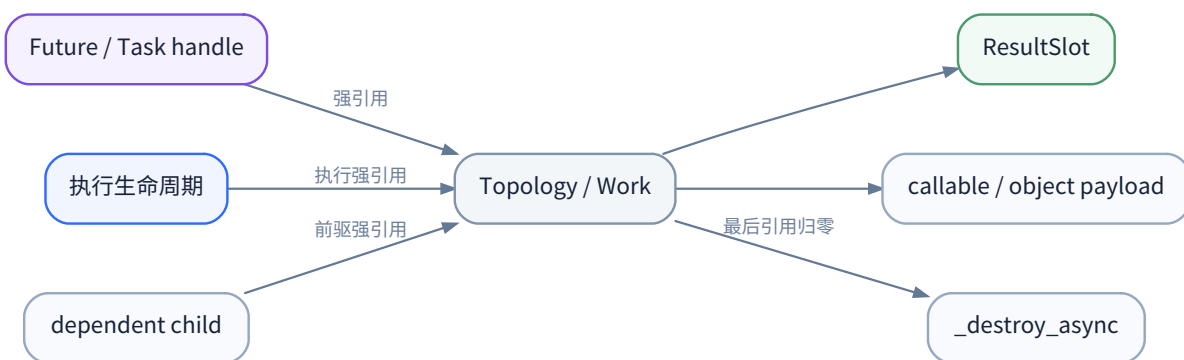


图 29 • Async Work 的句柄引用、执行引用、前驱引用与最终销毁关系

因此“Future 句柄析构”不等于“任务立即销毁”：任务可能仍在执行，也可能仍被后继依赖持有。相反，一个已完成任务如果仍被 Future 保存，也会继续保留结果槽供重复 `get()`。

44. 禁用异常

TaskflowLite 默认跟随编译器异常支持：

```
TFL_HAS_EXCEPTIONS  
TFL_ENABLE_EXCEPTIONS
```

显式关闭：

```
target_compile_definitions(app PRIVATE TFL_ENABLE_EXCEPTIONS=0)
```

也可以配合编译器真正禁用异常，例如 GCC/Clang `-fno-exceptions`、MSVC 对应 `no-exception` 配置。

44.1 关闭后的语义

`TFL_ENABLE_EXCEPTIONS=0` 时：

- `TFL_THROW(...)` 变成 `std::terminate()`；
- 框架异常 catch/propagation 路径被编译关闭；
- 非法 API 调用不能再靠 `try/catch` 恢复；
- 用户 callable 也应保持不抛异常；
- 同一程序所有使用 TaskflowLite 的翻译单元必须统一设置。

示例：

```
auto task = flow.emplace([&]() noexcept {  
    result = 42;  
});
```

“关闭异常”不是把错误自动变成返回码，而是选择一种更严格的“错误即终止/业务不抛”运行模型。

45. Debug 检查和关键宏

45.1 `TFL_ENABLE_ASSERT`

- Debug 默认开；
- Release 默认关；
- Release 强开时可能退化为 compiler assume，而不是普通运行期 assert。

45.2 TFL_ENABLE_WORK_EXECUTION_CHECK

Debug 默认启用，用于发现同一 Work 上一轮还没真正完成却再次进入执行的内部一致性错误。

45.3 TFL_CACHE_LINE_SIZE

用于高频并发数据的 `alignas` 隔离。不同平台有默认估计，也可在包含头文件前覆盖。

它可能影响类型布局，同一程序翻译单元必须一致。

45.4 TFL_ENABLE_TASK_POOL

控制 Work 对象池。Benchmark 或 allocator fault-injection 时可显式关闭以得到更可控的分配路径。

46. 线程安全与生命周期清单

这是生产代码最应该反复检查的一页。

46.1 Flow

- 执行时不修改；
- 不并发重复提交同一个可变图实例；
- 左值提交时 Flow 活到执行完成；
- 移动/析构前先确保没有执行引用。

46.2 Task / TaskObject

- 非拥有；
- `valid()` 不检测悬空；
- `erase/clear` 后不再使用旧句柄；
- object 引用不自带同步。

46.3 AsyncFuture / AsyncTask

- 不同句柄副本可以共享底层任务；
- **同一个句柄对象**不能一边 `reset/move`，一边从另一线程访问它自身的非原子成员；
- 创建延迟任务的 Executor 必须活过 `start` 与实际执行；
- 动态依赖的前驱句柄本身不能在读取其内部状态时被并发移动/重置。

46.4 Runtime / SubFlow / TaskGroup / Branch / Jump

- 栈绑定、回调绑定；
- 不保存；
- 不跨线程传递；
- 不在 callback 返回后使用。

46.5 Semaphore

- 任务只借用地址；
- 任务完成并释放资源前对象必须存活。

46.6 Observer / Handler

- 可能在多个 Worker 并发回调；
- 共享状态自己同步；
- WorkerHandler hook 必须 noexcept。

47. 常见错误与排查

47.1 “任务没有执行”

检查：

1. 只是构建了 `SubFlow`，是否忘了 `run()`；
2. `AsyncTask` 是否还处于 `Idle`，忘了 `start()`；
3. `Branch` 是否没有选择任何后继；
4. `Semaphore` 是否没有可用配额；
5. 动态后继是否还在等未完成 predecessor；
6. 图是否存在不受 `Jump` 控制的非法循环。

47.2 “单线程卡住”

检查是否在 Worker 里：

```
child.get();
```

而 `child` 只能由同一个 `Executor Worker` 推进。

优先改为：

```
rt.wait_until([&] { return child.done(); });  
child.get();
```

或组织成 `TaskGroup::wait/corun`。

47.3 “依赖前驱抛异常了，后继为什么还可能被调度”

因为动态依赖表达的是完成顺序，不是 exception short-circuit。

如果后继业务要求“前驱失败则后继失败”，在后继中显式：

```
predecessor.get();
```

47.4 “TaskGroup 里抛异常，离开作用域没有 catch 到”

析构 `noexcept` 只保证等待，不传播。

显式：

```
group.wait();
```

47.5 “Semaphore reset 失败”

当前有 waiter 时 `reset()` 返回 false。不要把正在参与任务调度的 Semaphore 当成可随时重配的配置对象。

47.6 “图第二次运行结果不对”

业务 callable 的成员状态、外部变量不会自动恢复。框架只恢复调度所需的运行时依赖计数/拓扑状态。

48. 内存分配失败与强保证边界

当前仓库专门提供 standalone allocator fault-injection 回归，覆盖：

- `AsyncTask::start` dependency storage;
- predecessor 动态 successor 注册;
- Payload replacement;
- Executor / Runtime / TaskGroup 异步提交。

这说明维护层已经把“提交途中分配失败不能留下不可恢复半状态”作为核心测试目标。

对应用层仍建议：

1. 不把 OOM 恢复作为正常业务控制流；
2. 对必须在资源极限下运行的系统，实际用你自己的编译器、allocator、`TFL_ENABLE_TASK_POOL` 配置跑仓库 allocation tests；
3. 不假设一个由多个分配步骤组成的高层批量操作天然具有事务回滚，除非对应 API 文档明确保证；
4. `FlowBuilder::emplace(Ts...)`、unchecked 多边插入等已经明确是逐项生效，不提供整个调用的全事务。

49. 测试、Sanitizer 与 CI

典型本地构建：

```
cmake -S . -B build \  
  -DCMAKE_BUILD_TYPE=Debug \  
  -DTFL_BUILD_TESTS=ON  
  
cmake --build build --parallel  
ctest --test-dir build --output-on-failure
```

49.1 ASan

```
cmake -S . -B build/asan \  
  -DTFL_BUILD_TESTS=ON \  
  -DTFL_SANITIZER=ASAN
```

GNU/Clang 路线同时配置 address + undefined。

49.2 TSan

```
cmake -S . -B build/tsan \  
  -DTFL_BUILD_TESTS=ON \  
  -DTFL_SANITIZER=TSAN
```

TSan 只支持配置好的 Unix GCC/Clang 路线；Windows 使用 ASan。

49.3 Header 独立编译

测试系统会把活动 core header 单独作为 translation unit 编译，避免 umbrella header 偶然掩盖缺失 include。

49.4 no-exception 测试

同时测试：

- 显式 `TFL_ENABLE_EXCEPTIONS=0` ；
- 编译器真正关闭异常。

错误路径通过独立进程验证 `terminate` 行为。

50. Benchmark 方法

构建：

```
cmake -S . -B build/bench \  
-DTFL_BUILD_BENCHMARKS=ON \  
-DCMAKE_BUILD_TYPE=Release  
  
cmake --build build/bench --parallel
```

性能对比要保证：

- 相同 Worker 数；
- 相同任务图；
- 相同工作量；
- 构图是否计入计时保持一致；
- Release 编译选项一致；
- allocator/object pool 配置一致；
- 不把 Observer、日志 I/O 混入调度开销；
- 小任务要多轮取中位数/分位数，而不是看单次。

对调度器优化最有价值的分层测量

1. Work 创建/销毁；
2. 单 Worker 串行 cache 接力；
3. 本地 queue push/pop；
4. steal；
5. shared overflow；
6. fan-out/fan-in；
7. Runtime child；
8. async dynamic dependency；
9. Semaphore contention；
10. idle -> wake latency。

51. 完整实战示例集

这一章不再重复解释 API 定义，而是回答“**实际代码怎么组合**”。为了控制篇幅，除特别标注外，代码块省略重复的 `#include <taskflowlite/taskflowlite.hpp>` 和 `main()` 外壳；把代码放入 `main()` 即可按同样方式使用。

当前手册提供 **31 组使用场景**，再加上仓库自身的 01-39 示例，形成从入门到内部调试的完整练习路径。

组别	示例	重点
静态图与控制流	1-9	DAG、批量构图、Placeholder、Branch、Jump、Module、TaskObject
异步	10-16	Future、异常、defer、依赖链、钻石依赖、跨 Executor、AsyncTaskObject
动态运行时	17-22	Runtime、corun、wait_until、SubFlow、TaskGroup、停止
工程能力	23-31	Semaphore、Observer、WorkerHandler、parallel-for、reduce、retry、编辑、D2、综合 Pipeline

51.1 示例 1：最小但完整的 fan-out / fan-in DAG

场景：初始化完成后并行执行两个步骤，二者都完成后再汇聚。

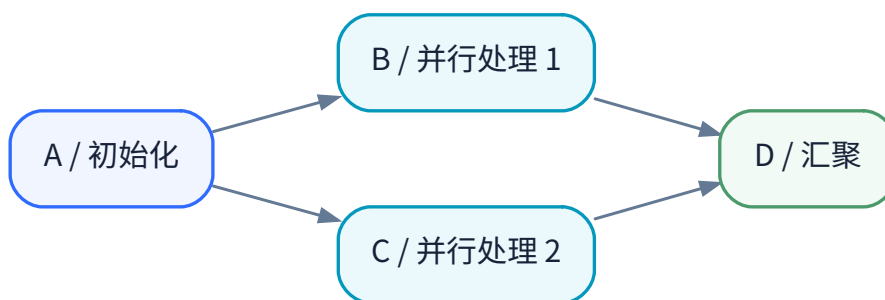


图 30 • $A \rightarrow (B, C) \rightarrow D$ 是最基本、也最常复用的任务图形状

```
int main() {
    tfl::Executor executor(4);
    tfl::Flow flow("basic-dag");

    std::atomic<int> value{0};
```

```

auto a = flow.emplace([& { value.store(10); }].name("init"));
auto b = flow.emplace([& { value.fetch_add(20); }].name("left"));
auto c = flow.emplace([& { value.fetch_add(12); }].name("right"));
auto d = flow.emplace([& { std::cout << value.load() << '\n'; }].name("merge"));

a.precede(b, c);
d.succeed(b, c);

executor.corun(flow);
}

```

关键点只有两个：`precede` 从当前节点指向后继；`succeed` 从当前节点反向声明前驱。`D` 只有等 `B/C` 两个 strong predecessor 都到达后才 ready。

51.2 示例 2：批量 `emplace` + `linearize`

场景：固定串行阶段很多时，不逐条写 `a.precede(b); b.precede(c);`。

```

tfl::Flow flow("linear");
auto [read, decode, infer, save] = flow.emplace(
    [] { std::cout << "read\n"; },
    [] { std::cout << "decode\n"; },
    [] { std::cout << "infer\n"; },
    [] { std::cout << "save\n"; }
);

flow.linearize(read, decode, infer, save);
executor.corun(flow);

```

已由上层保证图一定合法时可以使用 `linearize<false>(...)` 跳过拓扑检查；它仍可能因为边表扩容发生分配失败。

51.3 示例 3：Placeholder 先占拓扑，稍后绑定执行体

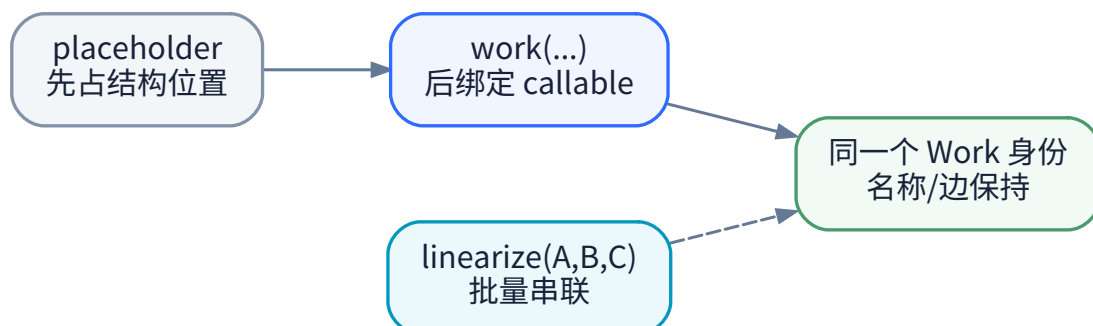


图 31 • Placeholder 适合先生成结构、后决定 callable 的构图器

```

tfl::Flow flow;
auto input = flow.emplace([] { std::cout << "input\n"; });
auto stage = flow.placeholder().name("replaceable");
auto sink = flow.emplace([] { std::cout << "sink\n"; });

flow.linearize(input, stage, sink);

stage.work([] { std::cout << "algorithm A\n"; });
executor.corun(flow);

stage.work([] { std::cout << "algorithm B\n"; });
executor.corun(flow);

```

节点身份和已有边没有变化，变的是 Work 的 payload。适合插件算法、运行模式切换、测试替身。

51.4 示例 4：Branch 实现真正的 if/else 路由

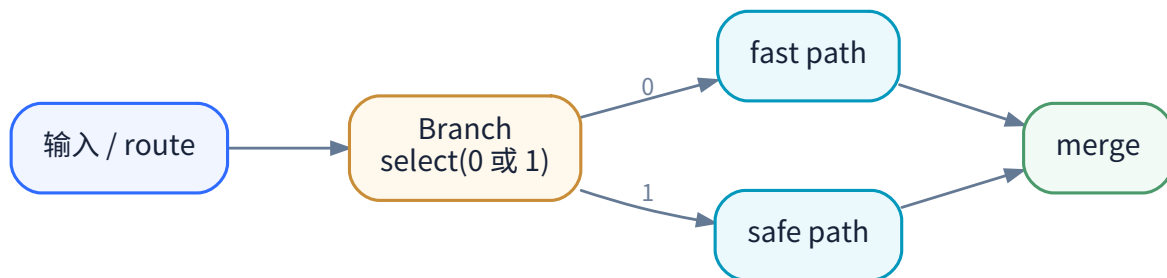


图 32 • Branch 只把正常 strong arrival 传播到被选路径

```

bool fast_mode = true;

auto route = flow.emplace([&](tfl::Branch& branch) {
    branch.select(fast_mode ? 0 : 1);
});

auto fast = flow.emplace([] { std::cout << "fast\n"; }).name("fast");
auto safe = flow.emplace([] { std::cout << "safe\n"; }).name("safe");
auto done = flow.emplace([] { std::cout << "done\n"; });

route.precede(fast, safe);
fast.precede(done);
safe.precede(done);
executor.corun(flow);

```

Branch 不是 Jump；目标如果还有其他强前驱，仍然必须等待那些前驱。

51.5 示例 5: MultiBranch 同时广播到多个后继

```
auto route = flow.emplace([](tfl::MultiBranch& branch) {
    branch.select(0, 2);          // 0、2 本轮执行
});

auto log    = flow.emplace([] { std::cout << "log\n"; });
auto debug  = flow.emplace([] { std::cout << "debug\n"; });
auto save   = flow.emplace([] { std::cout << "save\n"; });
route.precede(log, debug, save);
```

也可以使用 `select_all()` 或 `select_if(TaskView)`。它适合“同一事件触发多个可选处理器”，而不是循环回跳。

51.6 示例 6: Jump 构造受控 retry 循环

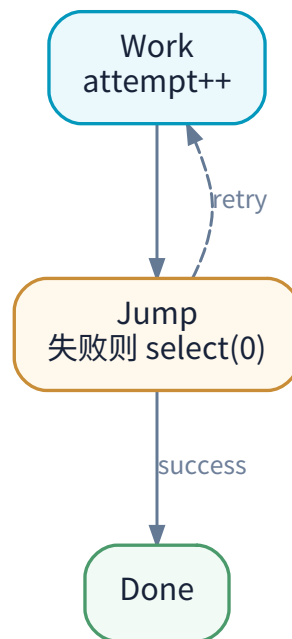


图 33 • Jump 的回边会强制重新激活目标节点

```
int attempts = 0;

std::function<bool()> succeed = [&] { return attempts >= 3; };

auto work = flow.emplace([&] {
    ++attempts;
    std::cout << "attempt " << attempts << '\n';
});

auto check = flow.emplace([&](tfl::Jump& jump) {
    if (!succeed()) jump.select(0); // successor 0 = work
});
```

```
auto done = flow.emplace([] { std::cout << "done\n"; });
work.precede(check);
check.precede(work, done);
executor.corun(flow);
```

Jump 会绕过目标普通 strong join 屏障，因此只应在明确需要重试、状态机回跳或受控循环时使用。

51.7 示例 7：Module 把成熟子流程复用 3 次

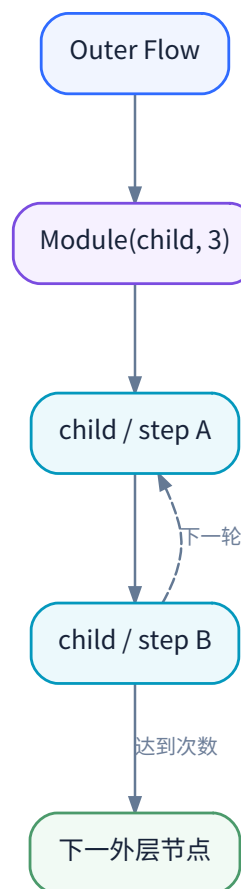


图 34 • Module 复用同一子图结构，不需要每轮重新构图

```
tfl::Flow child("child");
int calls = 0;
auto c1 = child.emplace([&] { ++calls; });
auto c2 = child.emplace([&] { ++calls; });
c1.precede(c2);

tfl::Flow outer("outer");
auto begin = outer.emplace([] { std::cout << "begin\n"; });
auto module = outer.emplace(child, 3ULL);
auto end = outer.emplace([&] { std::cout << "calls=" << calls << '\n'; });
outer.linearize(begin, module, end);
```

```
executor.corun(outer);
```

左值 `child` 是借用关系，必须活到模块最后一次执行结束。

51.8 示例 8：TaskObject 保存不可移动业务状态

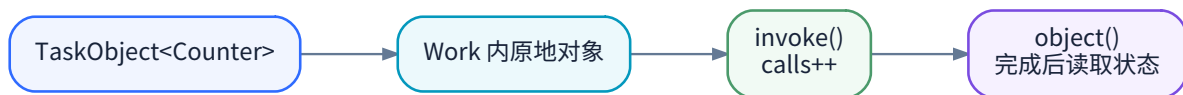


图 35 · 业务对象直接位于 Work payload 内，object() 只是借用引用

```
struct Counter {
    Counter() = default;
    Counter(const Counter&) = delete;
    Counter(Counter&&) = delete;

    int calls{};
    void operator()() { ++calls; }
};

auto node = flow.emplace_object<Counter>();
executor.corun(flow);
executor.corun(flow);
std::cout << node.object().calls << '\n';    // 2
```

不要在任务仍可能并发执行时无同步地读写 `object()`。

51.9 示例 9：保留拓扑，只替换对象实现

```
struct Algorithm {
    explicit Algorithm(int gain) : gain(gain) {}
    int gain{};
    void operator()() const { std::cout << gain << '\n'; }
};

auto stage = flow.placeholder().name("algorithm");
auto sink = flow.emplace([] {});
stage.precede(sink);

Algorithm& a = stage.work_object<Algorithm>(10);
executor.corun(flow);

Algorithm& b = stage.work_object<Algorithm>(20);
executor.corun(flow);
```

已有名称、边、Semaphore 和 Observer 都留在同一个 Work 上；旧对象引用在替换成功后失效。

51.10 示例 10: `async` 返回值与重复 `get()`

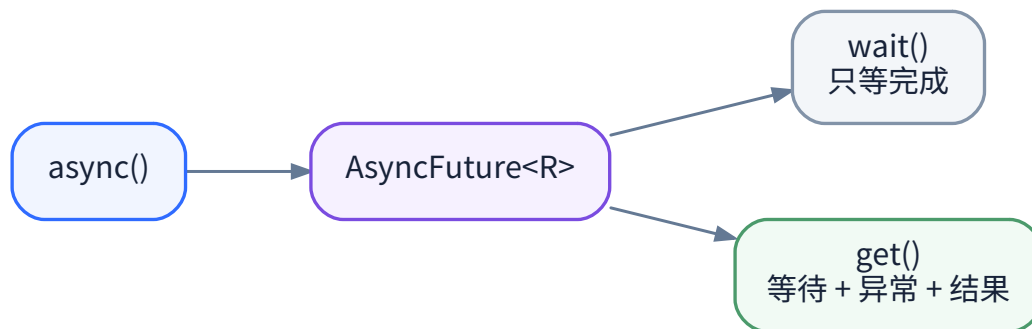


图 36 • `wait` 只等待，`get` 才负责结果与异常传播

```

auto future = executor.async([] { return 42; });
future.wait();
std::cout << future.get() << '\n';
std::cout << future.get() << '\n';    // 非消费式，可再次读取
  
```

对于值结果，`get()` 返回保存结果的 `const R&`；引用结果返回原引用；`void` 只完成等待和异常检查。

51.11 示例 11: 为什么 `wait()` 后仍然要 `get()`

```

auto future = executor.async([]() -> int {
    throw std::runtime_error("failed");
});

future.wait();    // 只表示完成，不抛任务异常

try {
    (void)future.get();    // 这里才重新抛出
} catch (const std::exception& e) {
    std::cout << e.what() << '\n';
}
  
```

这一区别对批量“先等待全部、再集中取结果”非常有用。

51.12 示例 12: `defer_async` 先创建、配置、再启动

```

auto task = executor.defer_async([] { return 7; });
task.name("delayed");

// 此时仍是 Idle，没有发布执行。
task.start();
std::cout << task.get() << '\n';
  
```

一次底层 AsyncTask 只能成功启动一次；`start()` 失败前后的状态规则应按当前异常契约处理，不要把一个任务当可重复提交对象。

51.13 示例 13：动态依赖链 Extract -> Transform -> Load

```
auto extract = executor.defer_async([] { return 21; });
auto transform = executor.defer_async([extract] {
    return extract.get() * 2;
});
auto load = executor.defer_async([transform] {
    std::cout << transform.get() << '\n';
});

extract.start();
transform.start(extract);
load.start(transform);
load.get();
```

依赖只保证“前驱完成后才能执行”；结果传递仍由 callable 显式捕获句柄并调用 `get()`。

51.14 示例 14：AsyncFuture 钻石依赖

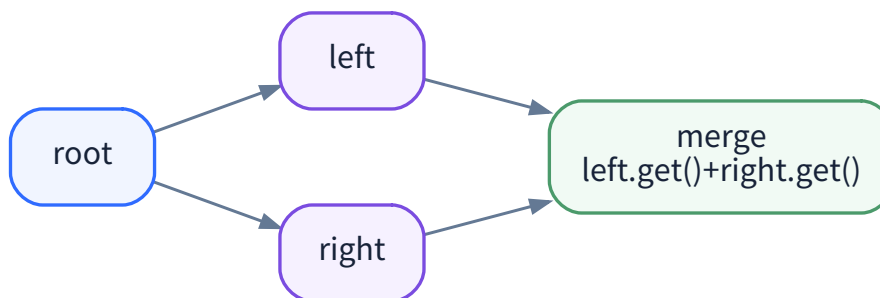


图 37 • root 完成后 left/right 并行，merge 等两者都完成

```
auto root = executor.async([] { return 1; });
auto left = executor.async([root] { return root.get() + 10; }, root);
auto right = executor.async([root] { return root.get() + 100; }, root);
auto merge = executor.async([left, right] {
    return left.get() + right.get();
}, left, right);

std::cout << merge.get() << '\n';    // 112
```

这是运行期异步图最常用的 fan-out / fan-in 形式。

51.15 示例 15：跨 Executor 的完成依赖

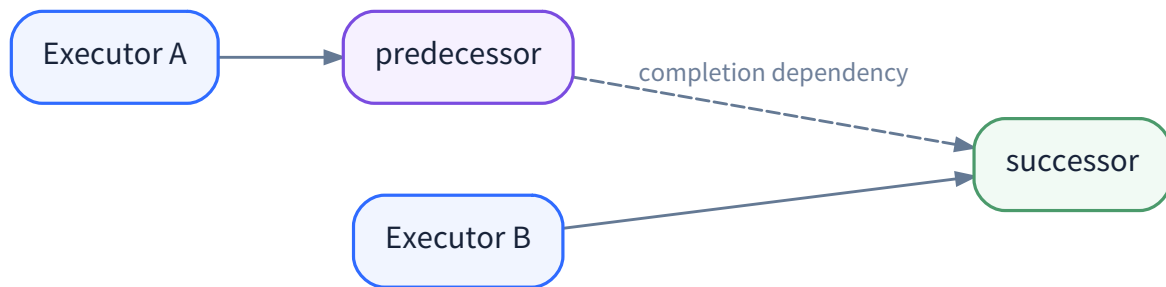


图 38 • 前驱在哪个 Executor 完成不影响后继在自己的 Executor 上被调度

```

tfl::Executor io_executor(2);
tfl::Executor cpu_executor(4);

auto io = io_executor.async([] { return 40; });
auto cpu = cpu_executor.async([io] { return io.get() + 2; }, io);

std::cout << cpu.get() << '\n';

```

依赖注册保存前驱强引用；后继 ready 后仍发布到后继所属 Executor。

51.16 示例 16：AsyncTaskObject 同时保存任务状态和结果

```

struct Job {
    explicit Job(int base) : base(base) {}
    int base{};
    int calls{};

    int operator()() {
        ++calls;
        return base + calls;
    }
};

auto task = executor.defer_async_object<Job>(41);
task.start();
std::cout << task.get() << '\n';
std::cout << task.object().calls << '\n';

```

适合“任务本身有稳定业务对象，同时又需要 Future 结果”的场景。

51.17 示例 17: Runtime 动态 fan-out, 然后协作等待

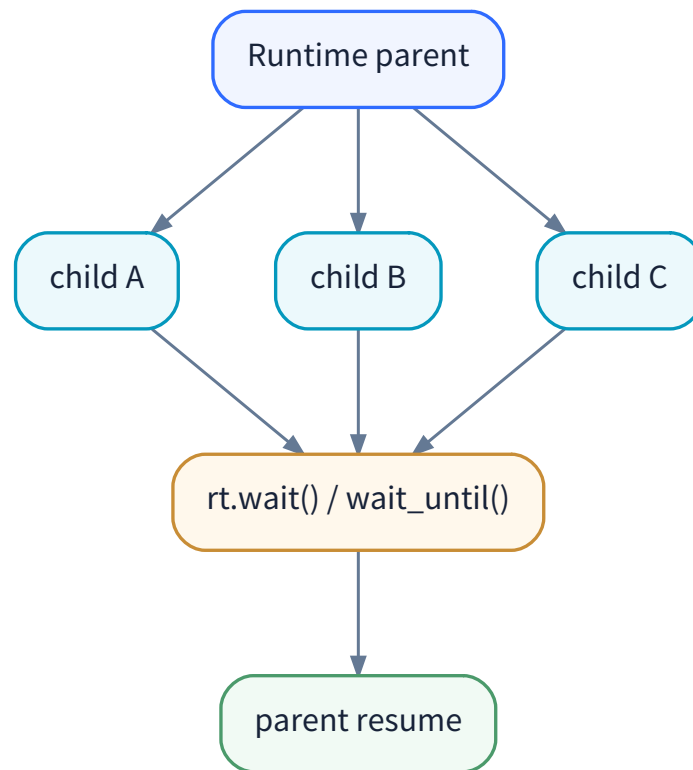


图 39 • Runtime 子任务计入父 Work 完成范围, 等待时 Worker 继续做其他 ready Work

```

auto parent = executor.async([](tfl::Runtime& rt) {
    std::array<std::atomic<int>, 4> out{};

    for (int i = 0; i < 4; ++i) {
        rt.silent_async([&, i] { out[i].store(i + 1); });
    }

    rt.wait();

    int sum = 0;
    for (auto& v : out) sum += v.load();
    return sum;
});

std::cout << parent.get() << '\n';           // 10
  
```

`rt.wait()` 是协作等待, 不是把 Worker 线程睡死。

51.18 示例 18: Runtime 中同步执行一个已有子图

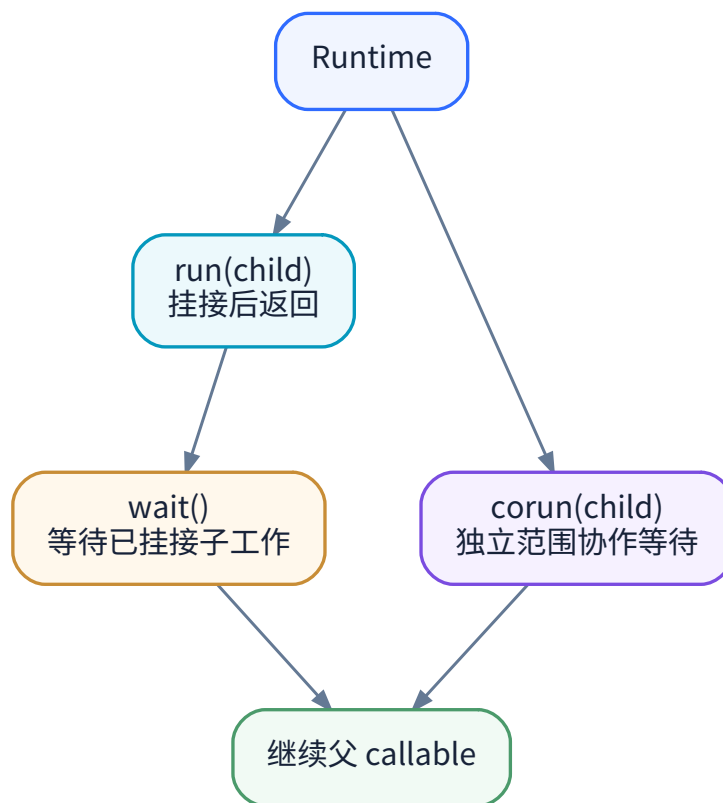


图 40 • run 负责挂接，corun 建立独立同步等待范围

```

tfl::Flow child("decode");
auto a = child.emplace([] { std::cout << "decode A\n"; });
auto b = child.emplace([] { std::cout << "decode B\n"; });
a.precede(b);

auto parent = executor.async([&](tfl::Runtime& rt) {
    rt.corun(child);           // 返回时 child 已完成
    std::cout << "after child\n";
});
parent.get();
  
```

如果使用 `rt.run(child)`，它把子图挂到当前父 Work 后立即返回；需要时再 `rt.wait()`。

51.19 示例 19: wait_until 等动态 Future 集合

```

auto parent = executor.async([&](tfl::Runtime& rt) {
    std::vector<tfl::AsyncFuture<int>> futures;
    for (int i = 0; i < 8; ++i) {
        futures.emplace_back(rt.async([i] { return i * i; }));
    }

    rt.wait_until([&] {
  
```

```

    return std::all_of(futures.begin(), futures.end(),
        [](const auto& f) { return f.done(); });
});

int sum = 0;
for (auto& f : futures) sum += f.get();
return sum;
});

```

它适合动态数量 Future，因为当前依赖接口不是“迭代器依赖列表”。

51.20 示例 20: SubFlow 根据输入动态构图

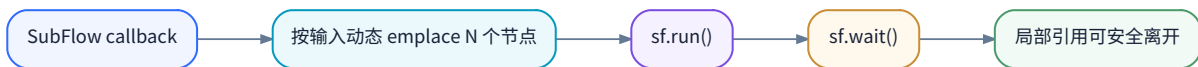


图 41 • SubFlow 必须显式 run；局部借用变量要在离开 callback 前 wait 完

```

std::vector<int> values{1, 2, 3, 4};

executor.async([&](tfl::SubFlow& sf) {
    for (int& value : values) {
        sf.emplace([&value] { value *= 10; });
    }

    sf.run();
    sf.wait();
}).get();

```

只 `emplace()` 不会自动执行；如果子任务捕获 callback 的局部变量，应在 callback 返回前 `wait()`。

51.21 示例 21: TaskGroup 管理局部批次

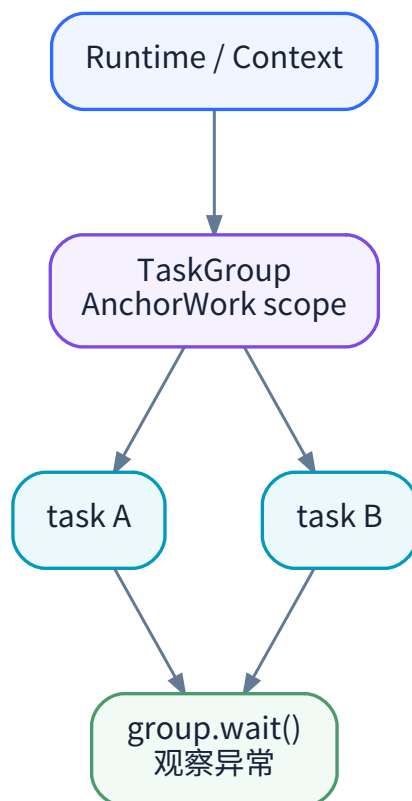


图 42 • TaskGroup 用 AnchorWork 聚合一组动态任务的完成范围

```
auto parent = executor.async([](tfl::Runtime& rt) {
    tfl::TaskGroup group(rt);

    auto a = group.async([] { return 20; });
    auto b = group.async([] { return 22; });

    group.wait();                // 显式等待才能在此观察组内异常
    return a.get() + b.get();
});

std::cout << parent.get() << '\n';
```

析构会等待未完成任务，但析构函数 `noexcept`，不要依赖离开作用域时传播子任务异常。

51.22 示例 22：协作式停止

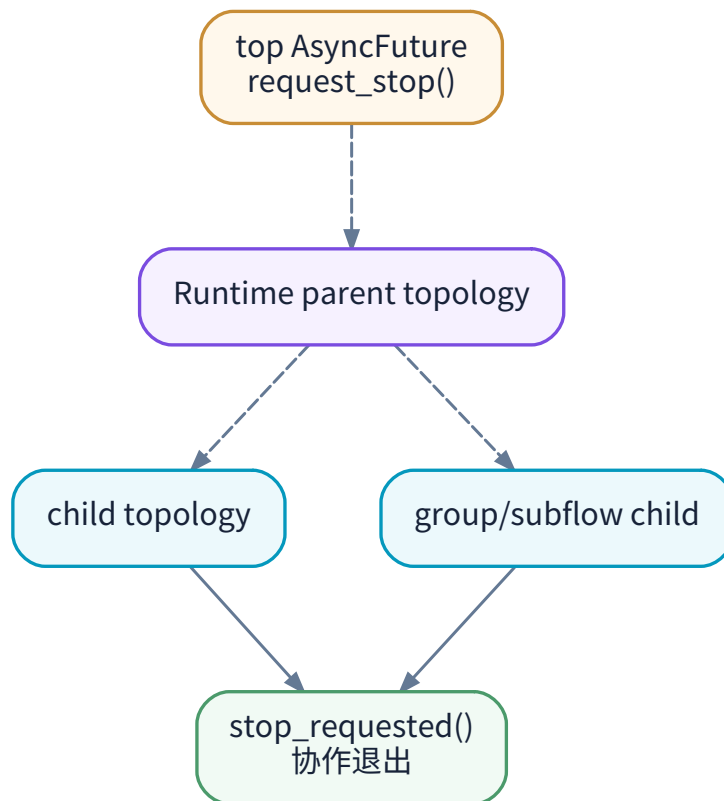


图 43 • request_stop 设置控制域；真正退出由任务中的 stop_requested 检查完成

```

auto job = executor.async([](tfl::Runtime& rt) {
    for (int i = 0; i < 1000; ++i) {
        if (rt.stop_requested()) return i;
        std::this_thread::yield();
    }
    return 1000;
});

job.request_stop();
std::cout << job.get() << '\n';

```

停止不是强制杀线程，也不会回滚已发生的副作用。

51.23 示例 23: Semaphore 把并行任务限制为固定资源数

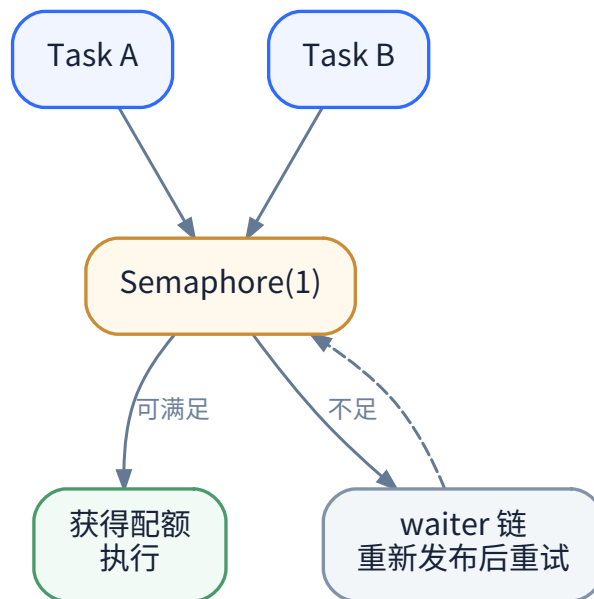


图 44 • 配额不足的 Work 进入 waiter 链，不阻塞 Worker

```

tfl::Semaphore gpu_slots(2);

for (int i = 0; i < 8; ++i) {
    flow.emplace([i] {
        std::cout << "gpu job " << i << '\n';
    }).acquire(gpu_slots).release(gpu_slots);
}

executor.corun(flow);
  
```

多资源 acquire 使用固定排序 + all-or-nothing 检查；不会先拿一部分再等待另一部分。

51.24 示例 24: 一个任务同时需要 GPU + 内存池

```

tfl::Semaphore gpu(1);
tfl::Semaphore buffers(4);

auto infer = flow.emplace([] { std::cout << "infer\n"; });
infer.acquire(gpu).acquire(buffers, 2);
infer.release(buffers, 2).release(gpu);
  
```

只有两个资源都满足时才统一扣减；任何一个不足都不会留下“部分占有”。

51.25 示例 25: Observer 做轻量任务计数



图 45 • Observer 包围单个 callable; WorkerHandler 包围 Worker OS 线程生命周期

```

struct CounterObserver final : tfl::TaskObserver {
    std::atomic<int> before{0};
    std::atomic<int> after{0};

    void on_before(tfl::WorkerView) override { ++before; }
    void on_after(tfl::WorkerView) override { ++after; }
};

auto task = flow.emplace([] {});
auto observer = task.register_observer<CounterObserver>();
executor.corun(flow);
std::cout << observer->before << ", " << observer->after << '\n';

```

Observer 可能被多个 Worker 并发回调，统计字段需要原子或其他同步。

51.26 示例 26: WorkerHandler 初始化每个 Worker 线程

```

struct Handler final : tfl::WorkerHandler {
    void on_start(tfl::Worker& worker) noexcept override {
        std::cout << "worker " << worker.id() << " start\n";
    }

    void on_stop(tfl::Worker& worker) noexcept override {
        std::cout << "worker " << worker.id() << " stop\n";
    }
};

Handler handler;
tfl::Executor executor(handler, 4);

```

这里适合设置线程名、CPU affinity、TLS、每线程 profiler; 不适合做每任务 tracing。

51.27 示例 27：手工 parallel-for 分块

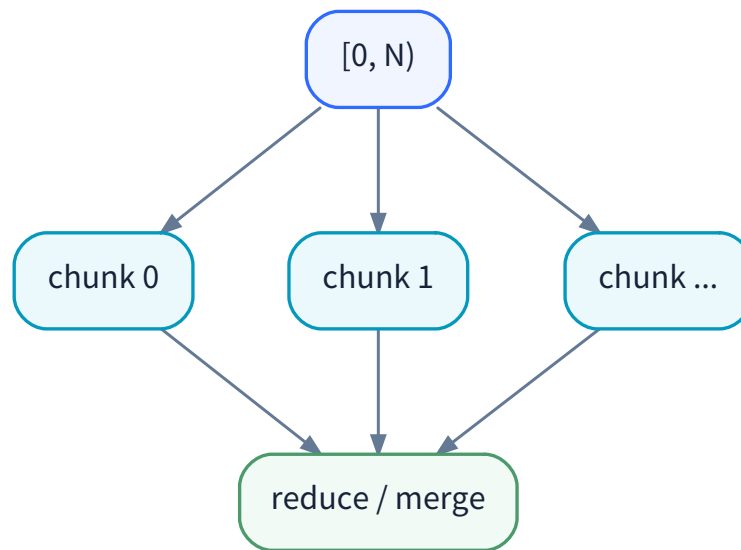


图 46 • 当前 core 没有独立 STL 算法层，常见做法是按 chunk 建任务或 Runtime 动态扇出

```

std::vector<int> data(1000, 1);
constexpr std::size_t chunk = 128;

tfl::Flow flow;
for (std::size_t first = 0; first < data.size(); first += chunk) {
    const std::size_t last = std::min(first + chunk, data.size());
    flow.emplace([&, first, last] {
        for (std::size_t i = first; i < last; ++i) data[i] *= 2;
    });
}

executor.corun(flow);

```

任务粒度要远大于调度开销；不要为了“并行”把每个元素都变成 Work。

51.28 示例 28：无锁共享写的 Map-Reduce

```

std::vector<int> data(1000, 1);
constexpr std::size_t chunk = 100;
std::array<int, 10> partial{};

tfl::Flow flow;
std::vector<tfl::Task> maps;

for (std::size_t k = 0; k < partial.size(); ++k) {
    maps.emplace_back(flow.emplace([&, k] {
        const auto first = k * chunk;
        const auto last = std::min(first + chunk, data.size());
        partial[k] = std::accumulate(data.begin() + first, data.begin() + last, 0);
    }));
}

```

```

}

int total = 0;
auto reduce = flow.emplace([&] {
    total = std::accumulate(partial.begin(), partial.end(), 0);
});
for (auto task : maps) task.precede(reduce);

executor.corun(flow);

```

每个 map 只写自己的 `partial[k]`，因此 reduce 前不需要原子累加。

51.29 示例 29: Retry + backoff 状态机

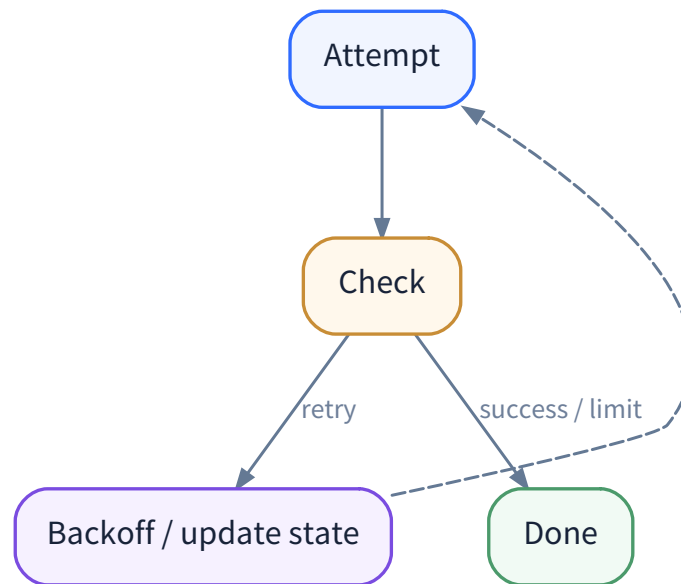


图 47 • 先更新重试状态，再由 Jump 显式回到 Attempt

```

int attempt = 0;
int backoff_ms = 1;

std::function<bool()> request = [&] { return ++attempt >= 4; };
bool ok = false;

auto call = flow.emplace([&] { ok = request(); });
auto backoff = flow.emplace([&] {
    if (!ok) {
        std::this_thread::sleep_for(std::chrono::milliseconds(backoff_ms));
        backoff_ms *= 2;
    }
});

auto gate = flow.emplace([&](tfl::Jump& jump) {
    if (!ok && attempt < 5) jump.select(0);
});

auto done = flow.emplace([] {});

```

```
flow.linearize(call, backoff, gate);
gate.precede(call, done);
executor.corun(flow);
```

把“是否继续”和“本轮副作用”分开，循环更容易审计。

51.30 示例 30：运行之间替换算法，但不重新构图



图 49 • task.work 只替换 payload；拓扑身份保持不变

```
int value = 0;
auto input = flow.emplace([&] { value = 10; });
auto algo = flow.placeholder().name("algorithm");
auto out = flow.emplace([&] { std::cout << value << '\n'; });
flow.linearize(input, algo, out);

algo.work([&] { value *= 2; });
executor.corun(flow); // 20

algo.work([&] { value += 5; });
executor.corun(flow); // 15
```

适合 A/B 算法、测试替身、运行模式切换；执行期间不要修改正在运行的图。

51.31 示例 31：完整视觉处理 Pipeline

这是一个更接近工程代码的组合：Capture 产生帧；Detect 与 Measure 并行；Decision 汇聚；Writer 受 I/O 配额限制；Detect 受 GPU 配额限制；整个 Flow 可以反复执行处理多帧。

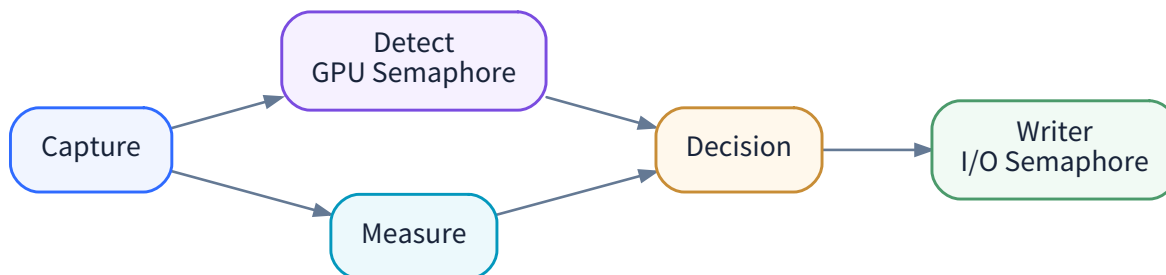


图 48 • Capture → Detect/Measure → Decision → Writer，并在任务层配置 GPU/I/O 资源约束

```

struct Frame { int id{}; };
struct Objects { int count{}; };
struct Measurement { int value{}; };

int main() {
    tfl::Executor executor(4);
    tfl::Semaphore gpu(1);
    tfl::Semaphore io(2);
    tfl::Flow flow("vision-pipeline");

    Frame frame;
    Objects objects;
    Measurement measurement;
    bool accepted = false;

    auto capture = flow.emplace([&] {
        ++frame.id;
    }).name("capture");

    auto detect = flow.emplace([&] {
        objects.count = frame.id % 5;
    }).name("detect").acquire(gpu).release(gpu);

    auto measure = flow.emplace([&] {
        measurement.value = frame.id * 10;
    }).name("measure");

    auto decision = flow.emplace([&] {
        accepted = objects.count > 0 && measurement.value >= 20;
    }).name("decision");

    auto writer = flow.emplace([&] {
        std::cout << "frame=" << frame.id
            << " objects=" << objects.count
            << " measurement=" << measurement.value
            << " accepted=" << accepted << '\n';
    }).name("writer").acquire(io).release(io);

    capture.precede(detect, measure);
    decision.succeed(detect, measure);
    decision.precede(writer);

    for (int i = 0; i < 5; ++i) {
        executor.corun(flow);
    }
}

```

这段代码展示了静态图最适合的场景：**结构固定、每轮数据变化、节点对象和拓扑都可复用**。如果节点数量、分支结构在每轮都根据输入变化，再切换到 Runtime/SubFlow。

51.31.1 什么时候把这个静态 Pipeline 改成 Runtime/SubFlow

变化	保持静态 Flow	改用 Runtime/SubFlow
每帧节点数量固定，只是数据变化	✓	
某阶段根据输入动态创建 N 个子任务		✓

变化	保持静态 Flow	改用 Runtime/SubFlow
只是在固定路径中选择一个分支	Branch	
需要重试/状态机回跳	Jump	Runtime 也可组合
子阶段已有成熟固定子图	Module	
动态 Future 数量直到运行时才知道		Runtime + wait_until

52. 39 个示例阅读路线

示例	主题
01	Basic DAG
02	Parallel
03	Loop
04	Runtime
05	Branch
06	Jump
07	Semaphore
08	SubFlow
09	Pipeline
10	Dump
11	Flow emplace
12	Loop workflow
13	Parallel reduce
14	AsyncTask chain
15	Observer
16	Error handling
17	Cancellation
18	Pipeline producer/consumer

示例	主题
19	Dependent async
20	Parallel for index
21	Observer tracing
22	State machine
23	Parallel reduce
24	Recursive Runtime
25	Retry/backoff
26	TaskGroup
27	Dynamic SubFlow
28	Task editing
29	AsyncFuture results
30	WorkerHandler
31	TaskObject
32	AsyncTaskObject
33	Unchecked task links
34	Dependency errors
35	BoundedQueue overflow
36	corun
37	no exceptions
38	Context and stop
39	Observer errors

推荐顺序

基础用户

01 -> 02 -> 05 -> 08 -> 09 -> 10

异步

```
14 -> 19 -> 29 -> 32 -> 34
```

动态执行

```
04 -> 24 -> 26 -> 27 -> 36 -> 38
```

调度器维护

```
33 -> 35 -> 36 -> 37 -> 38 -> 39
```

53. 公共 API 速查

这一节只列**用户层最常用公开入口**，便于按类型快速定位；模板约束和所有重载以对应头文件为准。

53.1 Flow / FlowBuilder

接口族	说明
<code>placeholder()</code>	创建无用户 callable 的结构节点
<code>emplace(callable)</code>	创建 Basic / Branch / MultiBranch / Jump / MultiJump / Runtime / SubFlow 节点
<code>emplace(graph [, num/predicate])</code>	创建 Module
<code>emplace_object<T>(args...)</code>	原地构造对象节点
<code>emplace(a,b,...)</code>	批量插入并返回 tuple
<code>linearize(...)</code> / <code>linearize<false>(...)</code>	串联任务
<code>erase(...)</code> / <code>clear()</code>	删除节点/清空图
<code>for_each(visitor)</code>	遍历所有任务
<code>size()/empty()/hash_value()</code>	图状态查询
<code>graph()</code>	获取底层 Graph

```
tfl::Flow flow("pipeline");
auto [a, b, c] = flow.emplace([] {}, [] {}, [] {});
```

```
flow.linearize(a, b, c);
executor.corun(flow);
```

53.2 Task / TaskView

```
// 身份和查询
bool valid() const;
std::string_view name() const;
TaskType type() const;
std::size_t num_predecessors() const;
std::size_t num_successors() const;
std::size_t num_acquires() const;
std::size_t num_releases() const;
std::size_t num_observers() const;

// 拓扑
precede(tasks...);
succeed(tasks...);
precede<false>(tasks...);
succeed<false>(tasks...);
remove_predecessor(...);
remove_successor(...);
clear_predecessors();
clear_successors();

// payload
work(callable_or_graph...);
work_object<T>(args...);

// 资源和观察
acquire(semaphore [, count]);
release(semaphore [, count]);
register_observer<Observer>(args...);
unregister_observer(observer);

dump(direction);
```

TaskView 提供相应只读查询和前驱/后继遍历，不允许改图。

53.3 Branch / MultiBranch / Jump / MultiJump

```
select(index...);
unselect(index...);
select_if(predicate);
reset();
operator()(index...);
operator[](index);    // proxy: controller[index] = true/false
size();

// Multi 版本
select_all();
unselect_all();
```

Branch = 正常依赖到达；Jump = 强制激活目标。

53.4 Executor

```
Executor(std::size_t workers = std::thread::hardware_concurrency());
Executor(WorkerHandler&, std::size_t workers = ...);

silent_async(task_or_graph, ...);
async(task_or_graph, deps...);
defer_async(task_or_graph, ...);
defer_async_object<T>(args...);

corun(graph);
wait_for_all();

num_workers();
num_waiters();
num_queues();
num_topologies();
```

最短选择规则

```
同步跑图      -> corun
立即异步有结果 -> async
立即异步无结果 -> silent_async
先配置再启动  -> defer_async / defer_async_object
```

53.5 AsyncFuture

```
valid();
running();
done();
wait();           // 只等待
get();           // 等待 + 重抛异常 + 结果

request_stop();
stop_requested();

use_count();
hash_value();
type();
dump();
reset();
```

`get()` 非消费式，可重复调用。

53.6 AsyncTask

除继承 `AsyncFuture<R>` 的结果/等待接口外，增加：

```

start(deps...);
name(value);

acquire(...);
release(...);
remove_acquire(...);
remove_release(...);
clear_acquires();
clear_releases();

for_each_acquire(...);
for_each_release(...);

register_observer<Observer>(...);
unregister_observer(...);

```

53.7 TaskObject / AsyncTaskObject<R,T>

```

auto task = flow.emplace_object<MyNode>(ctor_args...);
MyNode& object = task.object();

auto async = executor.defer_async_object<MyNode>(ctor_args...);
MyNode& state = async.object();
async.start();
async.get();

```

`object()` 只返回内部对象引用，不隐式等待、不加锁。

53.8 Context / Runtime

所有执行期 Context 都可访问：

```

worker();
executor();
stop_requested();
type();
name();

```

Runtime 增加：

```

silent_async(...);
async(...);
run(graph);           // 挂接后立即返回
corun(graph);         // 独立范围协作等待
wait();               // 等当前 Work 下已挂接子任务
wait_until(predicate); // 协作执行直到条件成立

```

53.9 SubFlow

SubFlow = FlowBuilder + Context :

```
emplace(...);
emplace_object<T>(...);
linearize(...);
run();
wait();
```

只构图不调用 `run()`，节点不会执行。

53.10 TaskGroup

```
TaskGroup(Context&);

silent_async(...);
async(...);
run(graph);
corun(graph);
wait();

request_stop();
stop_requested();
size();
```

析构会等待，但不会从析构函数传播子任务异常。

53.11 Semaphore

```
Semaphore(max);
Semaphore(max, current);

value();
max_value();
reset(max);
reset(max, current);
name();
name(value);
```

任务侧通过 `Task/AsyncTask::acquire/release` 配置使用。

53.12 TaskObserver / WorkerHandler

```
struct MyObserver : tfl::TaskObserver {
    void on_before(tfl::WorkerView) override;
    void on_after(tfl::WorkerView) override;
};
```

```
struct MyHandler : tfl::WorkerHandler {
    void on_start(tfl::Worker&) noexcept override;
    void on_stop(tfl::Worker&) noexcept override;
};
```

Observer 面向**任务执行事件**；WorkerHandler 面向**线程生命周期**。

53A. 常用场景配方

这些示例不引入新抽象，只把前面的 API 组合成直接可用的写法。

配方 1：两个前处理并行，一个后处理汇聚

```
auto a = flow.emplace([&] { preprocess_a(); });
auto b = flow.emplace([&] { preprocess_b(); });
auto c = flow.emplace([&] { postprocess(); });
c.succeed(a, b);
executor.corun(flow);
```

配方 2：异步任务链

```
auto load = executor.async([] { return load_data(); });
auto parse = executor.async([load] { return parse_data(load.get()); }, load);
auto save = executor.async([parse] { save_data(parse.get()); }, parse);
save.get();
```

配方 3：单 Worker 内安全等待子任务

```
auto parent = executor.async([](tfl::Runtime& rt) {
    auto child = rt.async([] { return 42; });
    rt.wait_until([&] { return child.done(); });
    return child.get();
});
```

配方 4：资源受限任务

```
tfl::Semaphore io(4);
for (int i = 0; i < 32; ++i) {
    flow.emplace([&, i] { do_io(i); }).acquire(io).release(io);
}
executor.corun(flow);
```

配方 5：局部任务组

```
auto task = executor.async([&](tfl::Runtime& rt) {
    tfl::TaskGroup group(rt);
    for (int i = 0; i < 8; ++i) group.silent_async([i] { work(i); });
    group.wait();
});
task.get();
```

配方 6：动态数量 SubFlow

```
executor.async([&](tfl::SubFlow& sf) {
    for (auto& item : items) sf.emplace([&item] { process(item); });
    sf.run();
    sf.wait();
}).get();
```

配方 7：可复用对象节点

```
auto node = flow.emplace_object<StatefulNode>(config);
executor.corun(flow);
read_result(node.object());
executor.corun(flow);
```

配方 8：同步嵌套子图

```
auto parent = executor.async([&](tfl::Runtime& rt) {
    rt.corun(child_flow);
    return 42;
});
```

54. 当前 **main** 相对旧 PDF 必须更新的重点

旧版手册的总体结构仍然有价值，但以 2026-09-29 当前 core 为准，至少要把以下内容按新语义理解：

1. **Executor::corun** 已成为正式同步图执行入口，并有独立完整回归测试。
2. **Context** 统一执行期 Worker/Executor/stop/type/name 访问。
3. **Runtime / TaskGroup** 的停止继承应按当前 parent Topology 链说明。
4. **TaskGroup** 析构只等待且 **noexcept**；观察异常必须显式 **wait()**。
5. **Semaphore** 使用当前全量 waiter 重新发布 + all-or-nothing 多资源获取协议；**reset** 有 waiter 时返回 **false**。

6. 共享调度以当前 `SharedWorkStack` 为准，不再用旧版共享 `UnboundedQueue` 心智模型。
7. 静态 `strong join` 按当前统一 `fetch_sub` + 完成时 `fetch_add(join_weight)` 恢复协议说明。
8. `Observer` 异常进入 `Work` 异常归档流程。
9. 无异常模式要覆盖显式 `opt-out` 和编译器真正禁用异常两种路径。
10. 示例已经扩展到 39 个，新增 `corun`、`no-exceptions`、`Context/stop`、`Observer errors`。
11. `Task/Async` 对象原地构造应覆盖 `work_object`、`emplace_object`、`defer_async_object` 当前接口。
12. 算法层只能按当前示例中的并行模式介绍，不能把尚未进入 `core` 的未来 API 写成正式能力。

55. 设计和使用原则总结

如果只记住十条：

1. `Flow` 拥有节点，`Task` 不拥有。
2. 图运行时不修改，也不要并发重复提交同一个图实例。
3. 同步跑一个图优先理解 `corun`。
4. `AsyncFuture::get()` 才传播异步异常；`wait()` 只等待。
5. `Async` 动态依赖只表达“完成先后”，不自动传播前驱结果和异常。
6. `Worker` 内等待动态工作优先使用 `Runtime/TaskGroup` 的协作等待。
7. `Branch` 遵守普通 `join`；`Jump` 强制激活，二者不是同一种控制流。
8. `Runtime/SubFlow/TaskGroup/Branch/Jump` 都是回调期借用对象，不能逃逸。
9. `Semaphore` 多资源获取是 `all-or-nothing`，拿不到资源的 `Work` 不阻塞 `Worker`。
10. 排查调度问题时沿着 `source -> schedule -> queue/shared -> invoke -> tear-down -> join_counter -> parent slot` 这条链看。

56. 源码索引

用户层首要文件：

```
taskflowlite/taskflowlite.hpp
taskflowlite/core/flow.hpp
taskflowlite/core/flow_builder.hpp
taskflowlite/core/task.hpp
taskflowlite/core/task_object.hpp
taskflowlite/core/executor.hpp
taskflowlite/core/async_future.hpp
taskflowlite/core/async_task.hpp
taskflowlite/core/async_task_object.hpp
taskflowlite/core/context.hpp
taskflowlite/core/runtime.hpp
taskflowlite/core/subflow.hpp
```

```
taskflowlite/core/task_group.hpp
taskflowlite/core/branch.hpp
taskflowlite/core/jump.hpp
taskflowlite/core/semaphore.hpp
taskflowlite/core/observer.hpp
taskflowlite/core/worker.hpp
```

理解内部调度：

```
taskflowlite/core/work.hpp
taskflowlite/core/work_invokers.hpp
taskflowlite/core/work_storage.hpp
taskflowlite/core/work_factory.hpp
taskflowlite/core/graph.hpp
taskflowlite/core/topology.hpp
taskflowlite/core/bounded_queue.hpp
taskflowlite/core/shared_work_stack.hpp
taskflowlite/core/notifier.hpp
taskflowlite/core/small_vector.hpp
taskflowlite/core/object_pool.hpp
taskflowlite/core/spin_mutex.hpp
```

57. 文档维护规则

以后 core 发生修改时，建议按以下顺序同步这份手册：

1. 检查 `taskflowlite/taskflowlite.hpp` 版本和公共 include；
2. 查看 `core` 新增/删除文件；
3. 核对 `FlowBuilder` / `Task` / `Executor` / `Runtime` / `TaskGroup` 公共声明；
4. 核对 `Work::invoke` 与 `Executor` tear-down；
5. 核对 `Topology::Control` 和停止/引用语义；
6. 核对队列和 `Notifier`；
7. 核对 `Semaphore`；
8. 核对异常/无异常宏；
9. 核对 `examples/README.md` 与示例数量；
10. 跑测试与 sanitizer；
11. 最后才更新 PDF。

这样可以避免“文档已经写成理想设计，但 core 其实还没实现”的问题。

附录 A / 一个较完整的组合示例

下面把静态图、Semaphore、Runtime、TaskGroup、异步结果和停止放在同一个示例里。

```

#include <taskflowlite/taskflowlite.hpp>
#include <atomic>
#include <iostream>

int main() {
    tfl::Executor executor(4);
    tfl::Semaphore device(2, "device");
    tfl::Flow flow{"batch"};

    std::atomic<int> completed{0};

    auto prepare = flow.emplace([] {
        // prepare batch
    }).name("prepare");

    auto dynamic = flow.emplace([&](tfl::Runtime& rt) {
        tfl::TaskGroup group(rt);

        auto a = group.async([&] {
            completed.fetch_add(1, std::memory_order_relaxed);
            return 20;
        });

        auto b = group.async([&] {
            completed.fetch_add(1, std::memory_order_relaxed);
            return 22;
        });

        group.wait();

        const int sum = a.get() + b.get();
        if (sum != 42) {
            std::terminate();
        }
    }).name("dynamic");

    auto finish = flow.emplace([&] {
        std::cout << "completed=" << completed.load() << '\n';
    }).name("finish");

    dynamic.acquire(device).release(device);

    prepare.precede(dynamic);
    dynamic.precede(finish);

    executor.corun(flow);
}

```

这个示例里：

- **Flow** 负责稳定业务阶段；
- **Runtime** 在某阶段内产生动态工作；
- **TaskGroup** 给动态工作局部分组；
- **group.wait()** 是协作等待且传播组内异常；
- **Semaphore** 控制 **dynamic** 阶段使用设备的并发许可；
- 最外层 **corun** 给调用者一个明确同步完成边界。

附录 B / 选择接口的决策表

场景	首选
静态已知依赖	<code>Flow + Task</code>
立即异步计算	<code>Executor::async</code>
先配置再启动	<code>defer_async + AsyncTask::start</code>
不关心结果	<code>silent_async</code>
同步执行一个图	<code>Executor::corun</code>
Worker 内动态产生任务	<code>Runtime</code>
Worker 内动态构图	<code>SubFlow</code>
一组动态任务要局部等待/停止	<code>TaskGroup</code>
条件选择正常路径	<code>Branch/MultiBranch</code>
强制激活/循环	<code>Jump/MultiJump</code>
有状态不可移动 callable	<code>TaskObject / AsyncTaskObject</code>
限制稀缺资源并发	<code>Semaphore</code>
跟踪任务执行	<code>TaskObserver</code>
设置线程名/亲和性	<code>WorkerHandler</code>
查看任务结构	<code>dump(D2)</code>